Software Evolution and Maintenance

A Practitioner's Approach

Chapter 3
Evolution and Maintenance Models

© Tripathy & Naik

Outline of the Chapter

3.1 General Idea

3.2 Reuse Oriented Model

3.3 The Staged Model for CSS

3.4 The Staged Model for FLOSS

3.5 Change Mini-Cycle Model

3.6 IEEE/EIA 1219 Maintenance Process

3.7 ISO/IEC 14764 Maintenance Process

3.8 Software Configuration Management

3.8.1 Brief History

3.8.2 SCM Spectrum of Functionality

3.8.3 SCM Process

3.9 Change Request Workflow

© Tripathy & Naik

3.1 General Idea

- One traditional software development life cycle (SDLC) is shown in Figure, which comprises two discrete phases, namely:
  - development and
  - maintenance
- Maintenance approaching two-thirds of the product life-span.

- The percentages in Figure 3.1 indicate relative costs.

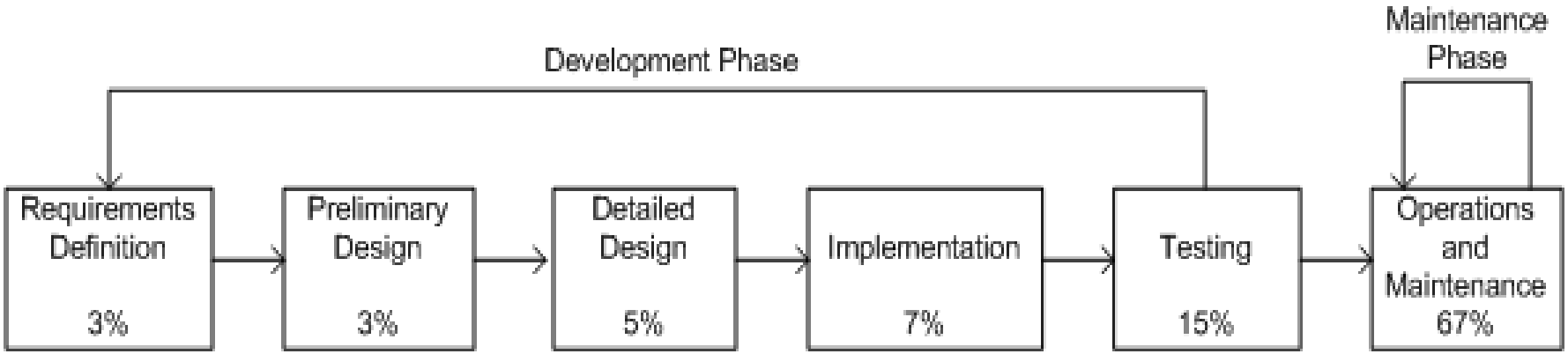


Figure 3.1: Traditional SDLC model ©Wiley, 1988

© Tripathy & Naik

3.1 General Idea

Software maintenance has unique characteristics:

- **Constraints of an existing system:** Maintenance is performed on an operational system. Therefore, all modifications must be compatible with the constraints of the existing architecture, design, and code.

- **Shorter time frame:** A maintenance activity may span from a few hours to a few months, whereas software development may span one or more years.

- **Available test data:** In software development, test cases are designed from scratch, whereas software maintenance can select a subset of these test cases and execute them as regression tests.

Software maintenance should have its own Software Maintenance Life Cycle (SMLC) model as it involves many unique activities.

© Tripathy & Naik

3.2 Reuse Oriented Models

- One obtains a new version of an old system by modifying one or several components of the old system and possibly adding new components.
- Based on this concept, three process models for maintenance have been proposed by Basili:
  - **Quick fix model:** In this model, necessary changes are quickly made to the code and then to the accompanying documentation.
  - **Iterative enhancement model:** In this model, first changes are made to the highest level documents. Eventually, changes are propagated down to the code level.
  - **Full reuse model:** In this model, a new system is built from components of the old system and others available in the repository.

© Tripathy & Naik

3.2 Reuse Oriented Models

In Quick Fix Model, as illustrated in Figure 3.2

(i) source code is modified to fix the problem;

(ii) necessary changes are made to the relevant documents; and

(iii) the new code is recompiled to produce a new version.

Often changes to the source code are made with no prior investigation such as analysis of impact of the changes, ripple effects of the changes, and regression testing.

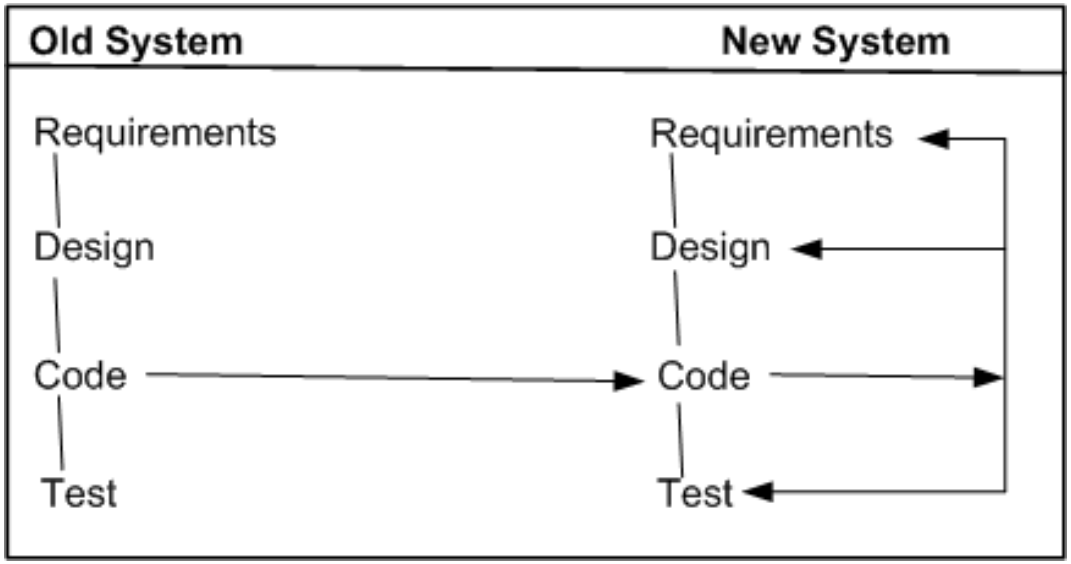


Figure 3.2 The quick fix model ©IEEE, 1990

© Tripathy & Naik

3.2 Reuse Oriented Models

Iterative Vs. Incremental

- The terms **iteration** and **increment** are liberally used when discussing iterative and incremental development.
- However, they are not synonyms in the field of software engineering.
- **Iteration** implies that a process is basically cyclic, thereby meaning that the activities of the process are repeatedly executed in a structured manner.
- **Iterative** development is based on scheduling strategies in which time is set aside to improve and revise parts of the system under development.
- **Incremental** development is based on staging and scheduling strategies in which parts of the system are developed at different times and/or paces, and integrated as they are completed.

© Tripathy & Naik

3.2 Reuse Oriented Models

The **iterative enhancement model**, explained in Figure 3.3, shows how changes flow from the very top level documents to the lowest-level documents.

The model works as follows:

- It begins with the existing system's artifacts, namely, requirements, design, code, test, and analysis documents.
- It revises the highest-level documents affected by the changes and propagates the changes down through the lower-level documents.
- The model allows maintainers to redesign the system, based on the analysis of the existing system.

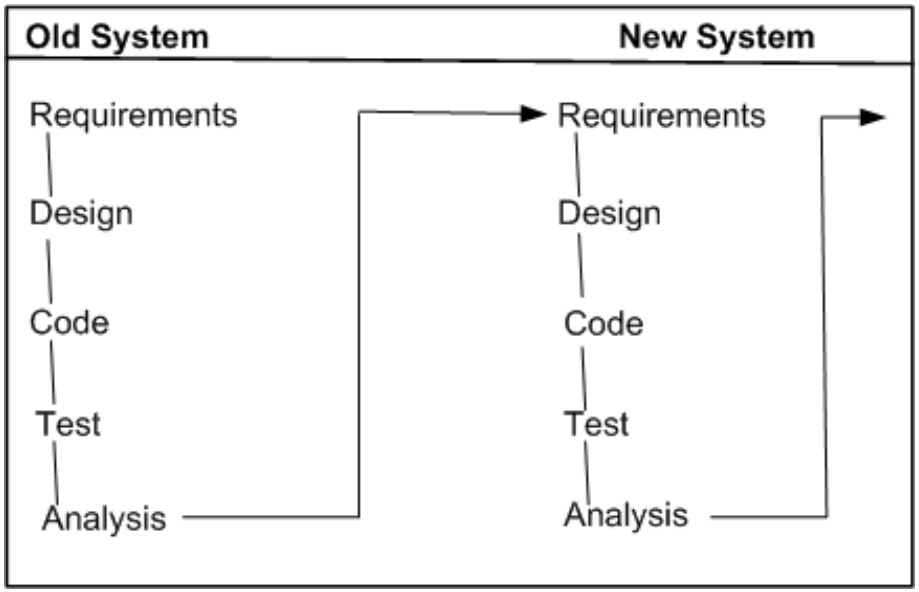


Figure 3.3 The iterative enhancement model ©IEEE, 1990

© Tripathy & Naik

3.2 Reuse Oriented Models

- The main assumption in this model is the availability of a repository of artifacts describing the earlier versions of the systems.
- In the **full reuse model**, reuse is explicit and the following activities are performed:
  - identify the components of the old system that are candidates for reuse
  - understand the identified system components.
  - modify the old system components to support the new requirements.
  - integrate the modified components to form the newly developed system.

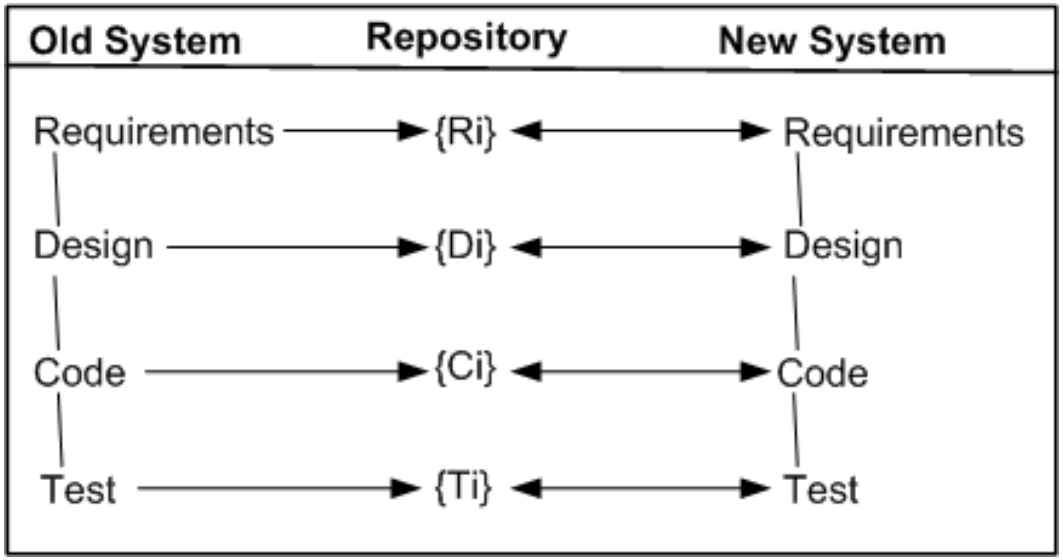


Figure 3.4 The full reuse model ©IEEE, 1990

© Tripathy & Naik

3.3 The Staged Model for CSS

- Rajlich and Bennett have presented a descriptive model of software evolution called the staged model of maintenance and evolution.
- Its primary objective is at improving understanding of how long-lived software evolves, rather than aiding in its management.
- Their model divides the lifespan of a typical system into four stages:
  - Initial development – The first delivered version is produced. Knowledge about the system is fresh and constantly changing. In fact, change is the rule rather than the exception. Eventually, an architecture emerge and stabilizes.
  - Evolution – Simple changes are easily performed and more major changes are possible too, albeit the cost and risk are now greater than in the previous stage. Knowledge about the system is still good, although many of the original developers will have moved on. For many systems, most of its lifespan is spent in this phase.
  - Servicing – The system is no longer a key asset for the developers, who concentrate mainly on maintenance tasks rather than architectural or functional change. Knowledge about the system has lessened and the effects of change have become harder to predict. The costs and risks of change have increased significantly.
  - Phase out – It has been decided to replace or eliminate the system entirely, either because the costs of maintaining the old system have become prohibitive or because there is a newer solution waiting in the pipeline. An exit strategy is devised and implemented, often involving techniques such as wrapping and data migration. Ultimately, the system is shut down.

© Tripathy & Naik

3.3 The Staged Model for CSS

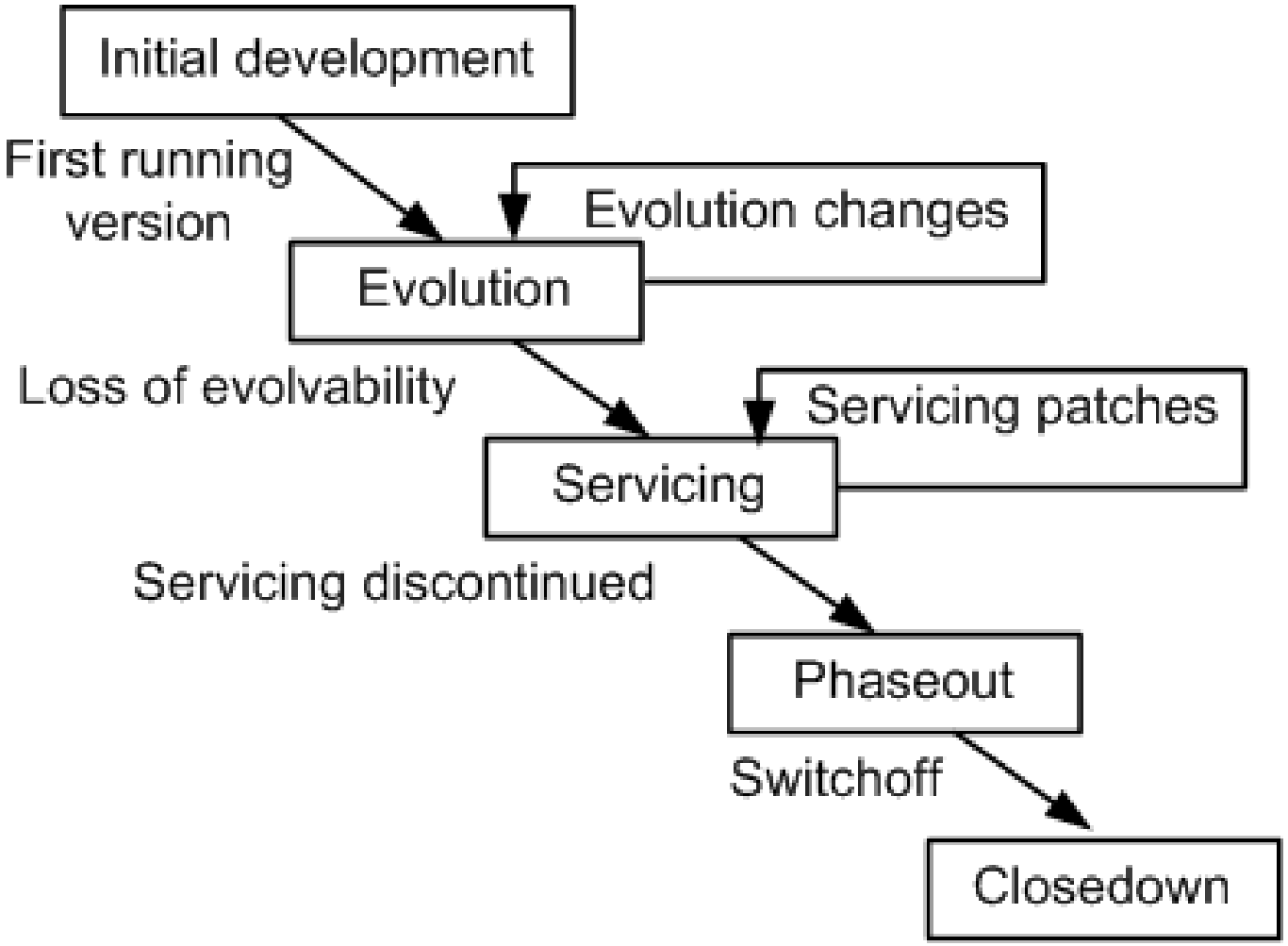


Figure 3.5 The simple staged model for the CSS life cycle ©IEEE, 2000

3.3 The Staged Model for CSS

The evolution process is the backbone of the model.

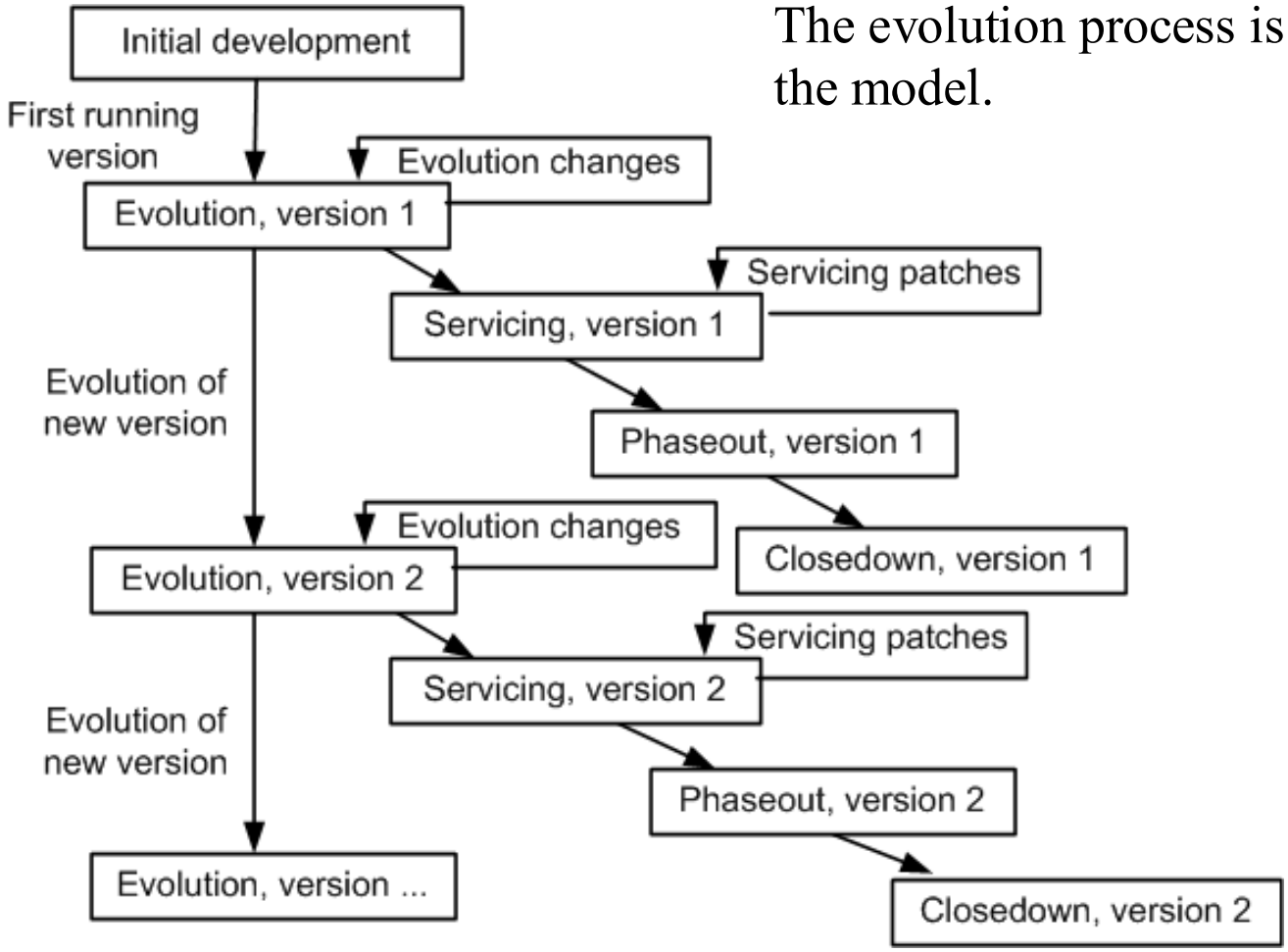


Figure 3.6 The versioned staged model for the CSS life cycle ©IEEE, 2000

© Tripathy & Naik

3.4 The Staged Model for FLOSS

- Three major differences are identified between CSS systems and FLOSS systems.
- In Figure 3.7, the rectangle with the label "Initial development" has been visually highlighted because it can be the only initial development stage in the evolution of FLOSS systems. In other words, it does not have any evolution track for FLOSS system.
- With some systems that were analyzed, after a transition from Evolution to Servicing, a new period of evolution was observed. This possibility is depicted in Figure as a broken arc from the Servicing stage to the Evolution stage.

- In general, the active developers of FLOSS systems get frequently replaced by new developers. Therefore, the dashed line in Figure exhibits this possibility of a transition from Phaseout stage to Evolution stage.

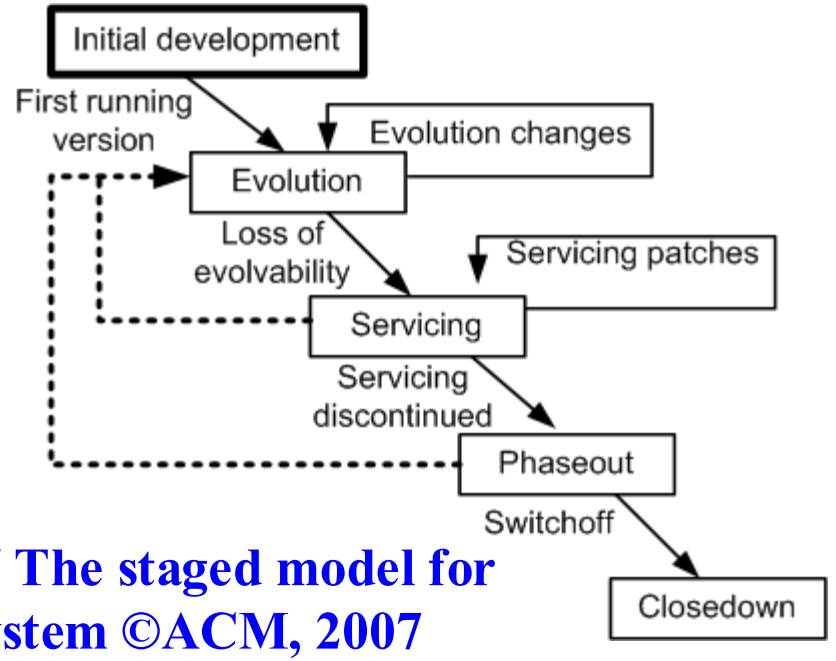


Figure 3.7 The staged model for FLOSS system ©ACM, 2007

© Tripathy & Naik

3.5 Change Mini-Cycle Model

Software change is a process that may introduce new requirements to the existing system, or may need to alter the software system if requirements are not correctly implemented. In order to capture this, an evolutionary model known as change mini-cycle was proposed by Yau et al. in the late seventies.

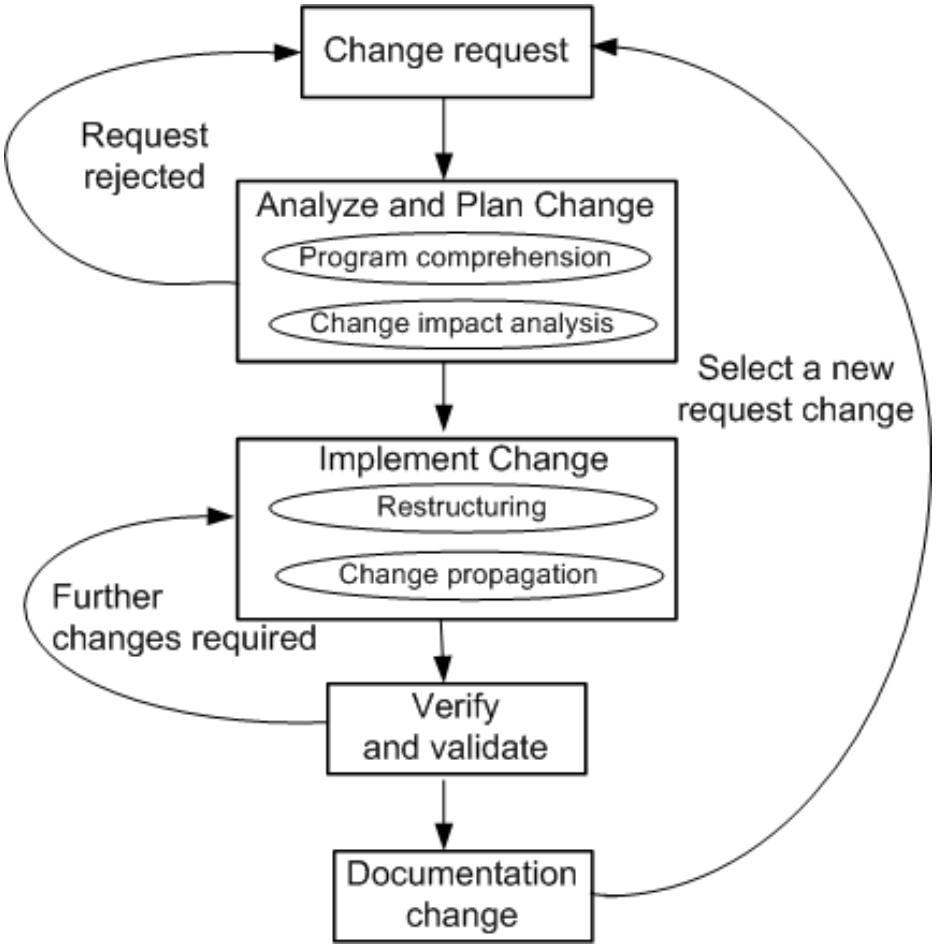


Figure 3.8 The change mini-cycle ©Springer, 2008

© Tripathy & Naik

3.6/3.7 Software Maintenance Standard

- IEEE and ISO have both addressed s/w maintenance processes.
- IEEE/EIA 1219 and ISO/IEC 14764 as a part of ISO/IEC12207 (life cycle process).
- IEEE/EIA 1219 organizes the maintenance process in seven phases:
  - problem identification, analysis, design, implementation, system test, acceptance test and delivery.
- ISO/IEC 14764 describes s/w maintenance as an iterative process for managing and executing software maintenance activities.
- The activities which comprise the maintenance process are:
  - process implementation, problem and modification analysis, modification implementation, maintenance review/acceptance, migration and retirement.
- Each of these maintenance activity is made up of tasks. Each task describes a specific action with inputs and outputs.

© Tripathy & Naik

Notes

- The standard ISO/IEC/IEEE 14764-2006 is the result of the harmonization of previous edition of the ISO/IEC 14764 and the IEEE Std 1219-1998
- Details the Software Maintenance Process as included in the standard ISO-IEC 12207 on the processes of the software life cycle
  - The standard ISO-IEC 12207-1995 has been revised by a joint task force of ISO/IEC and IEEE, resulting in the ISO/IEC/IEEE standard 12207-2008
- Latest version of the standard ISO/IEC-IEEE 14764 in 2022
  - Aligning to the new version (2017) of the standard ISO/IEC/IEEE 12207
  - Adding new type of enhancement: additive maintenance

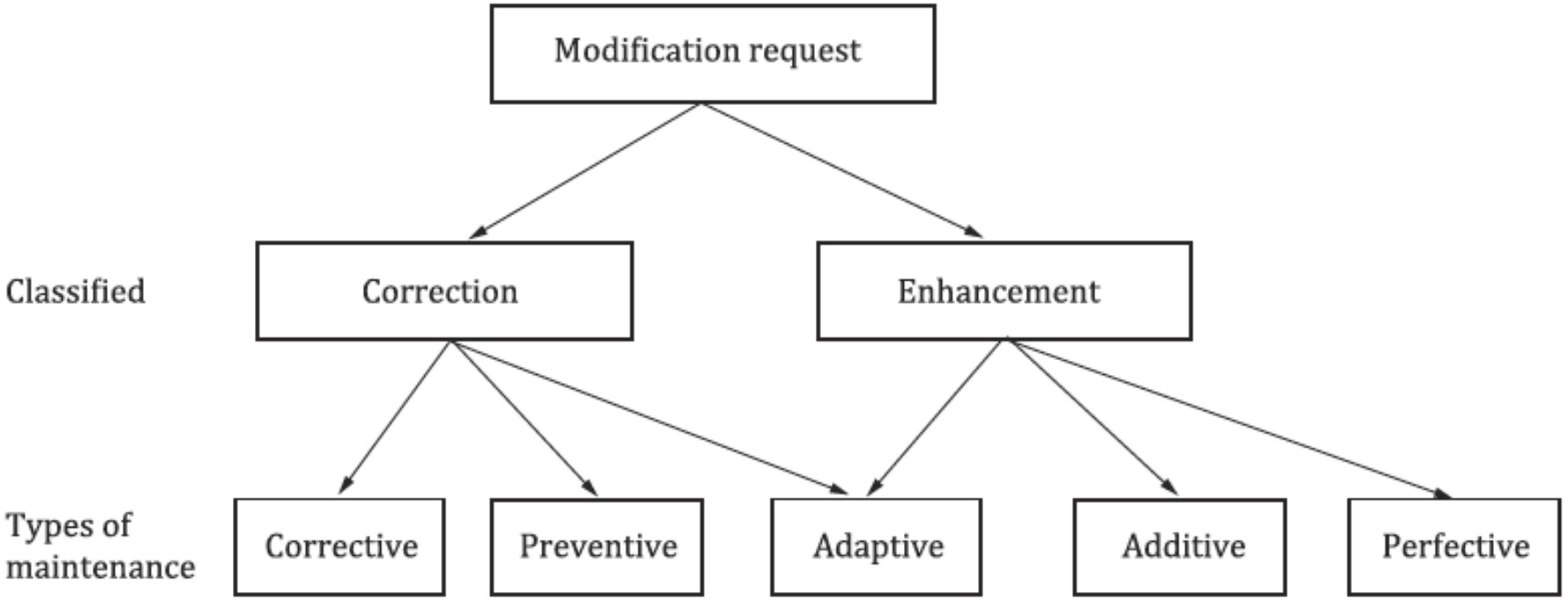
Definition: Software Maintenance

- *IEEE Std. 610.12-1990, IEEE Std. 1219-1998*
- Software maintenance is the modification of a software product or a component, after delivery, for the purpose of
  - to correct errors/failures/bugs
  - to improve system attributes such as performance
  - to adapt to a new or changed environment

Software Maintenance: A more general definition (std ISO/IEC & IEEE 14764)

- The totality of activities required to provide cost-effective support to a software system.
- Activities are performed during the predelivery stage as well as the postdelivery stage.
  - Predelivery activities include planning for the postdelivery operations, supportability, and logistics determination.
  - Postdelivery activities include software modifications, training, and operating a help desk.

Types of software maintenance



3.6 IEEE/EIA 1219 Maintenance Process

The standard focuses on a seven-phases:

- Problem Identification.
- Analysis.
- Design.
- Implementation.
- System Test.
- Acceptance Test.
- Delivery.

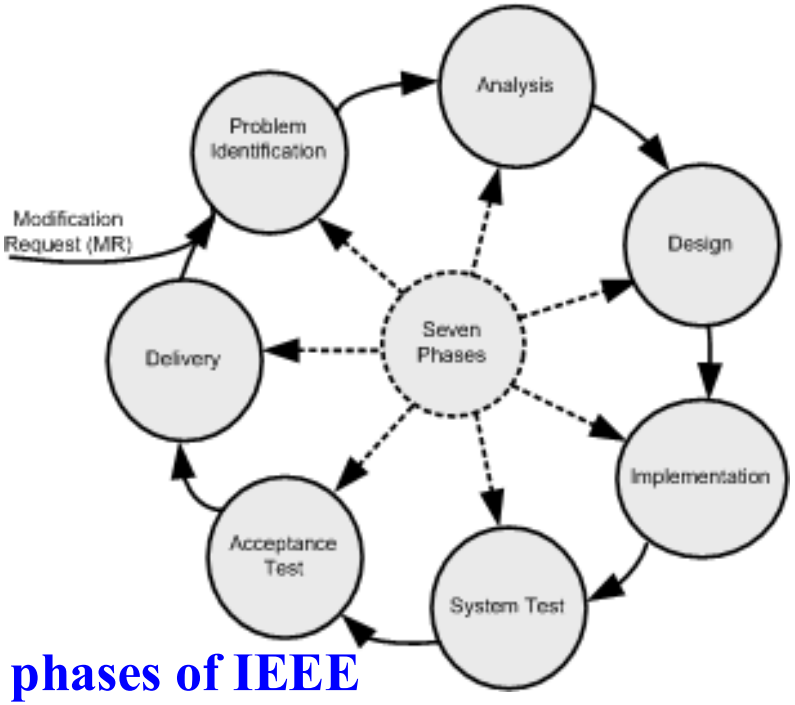


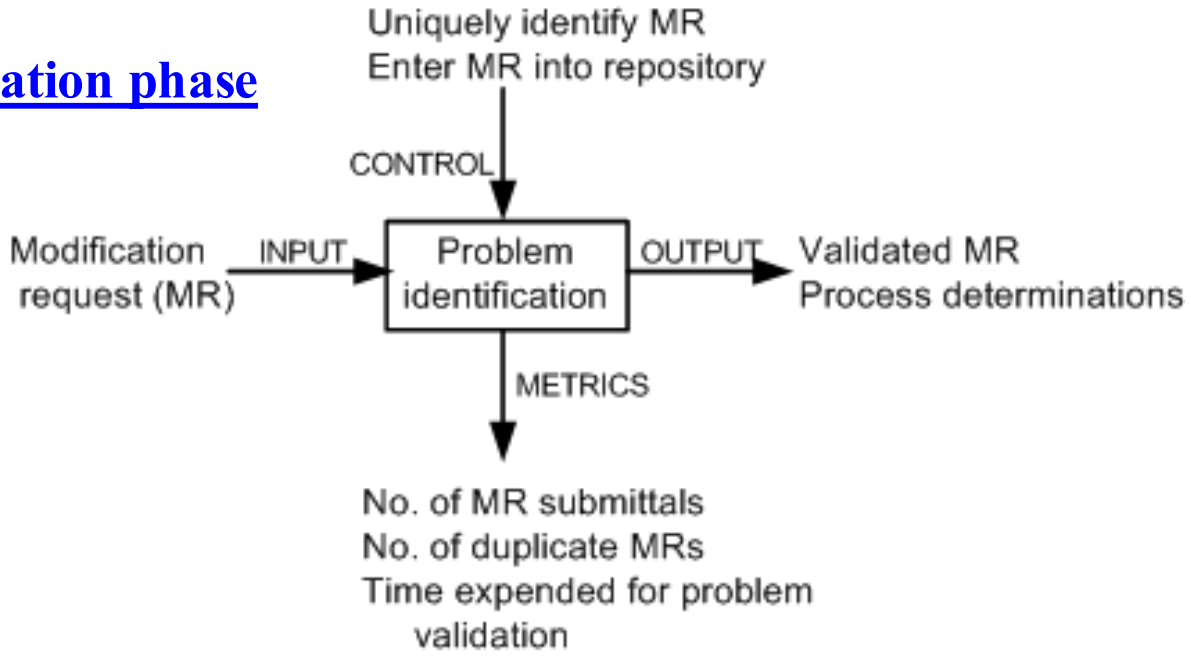
Figure 3.9 Seven phases of IEEE maintenance process ©IEEE, 2004

Each of the seven activities has five associated attributes as follows:

- **Activity definition:** This refers to the implementation process of the activity.
- **Input:** This refers to the items that are required as input to the activity.
- **Output:** This refers to the items that are produced by the activity.
- **Control:** This refers to those items that provide control over the activity.
- **Metrics:** This refers to the items that are measured during the execution of the activity.

3.6 IEEE/EIA 1219 Maintenance Process

Figure 3.10 Problem identification phase

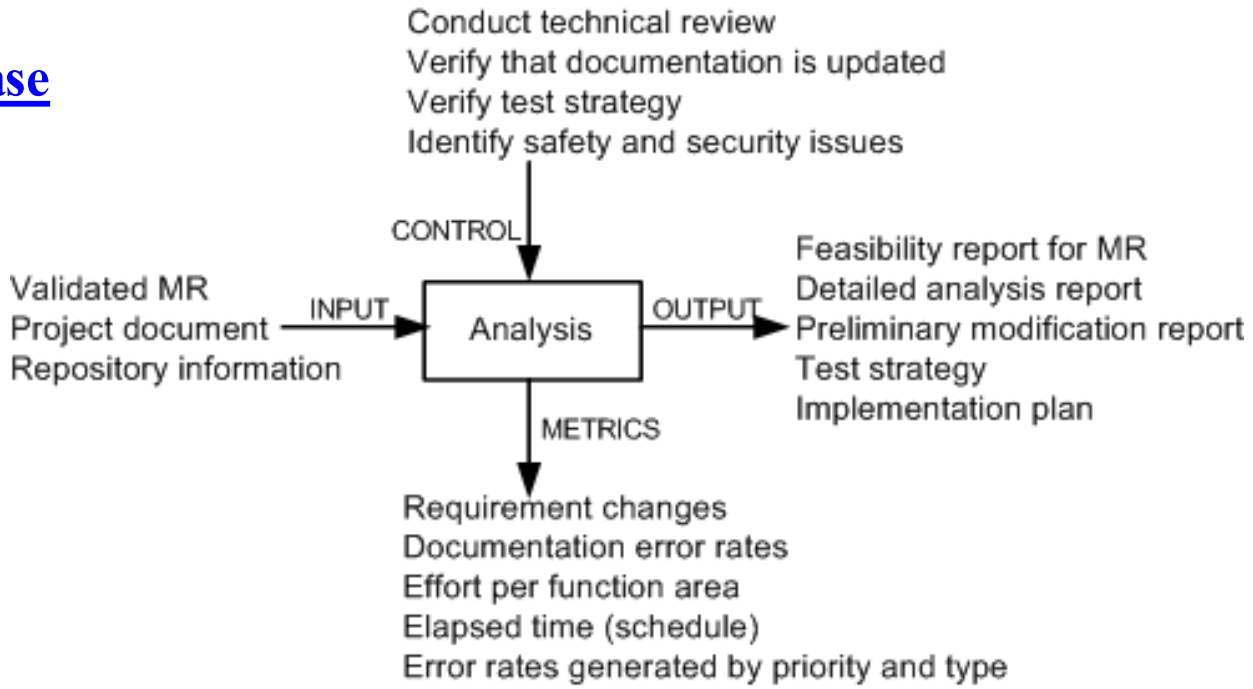


- A request for change to the software is normally made by the users of the software system or the customers, and it starts the maintenance process.
- The request for change (CR) is submitted in the form of a modification request (MR) for a correction or for an enhancement. MR & CR are used interchangeably.
- Activities included in this phase are as follows:
  (i) reject or accept the MR,
  (ii) identify and estimate the resources needed to change the system; and
  (iii) put the MR in a batch of changes scheduled for implementation.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

Figure 3.11 Analysis phase



The process is viewed to have two major components:

(i) feasibility analysis.

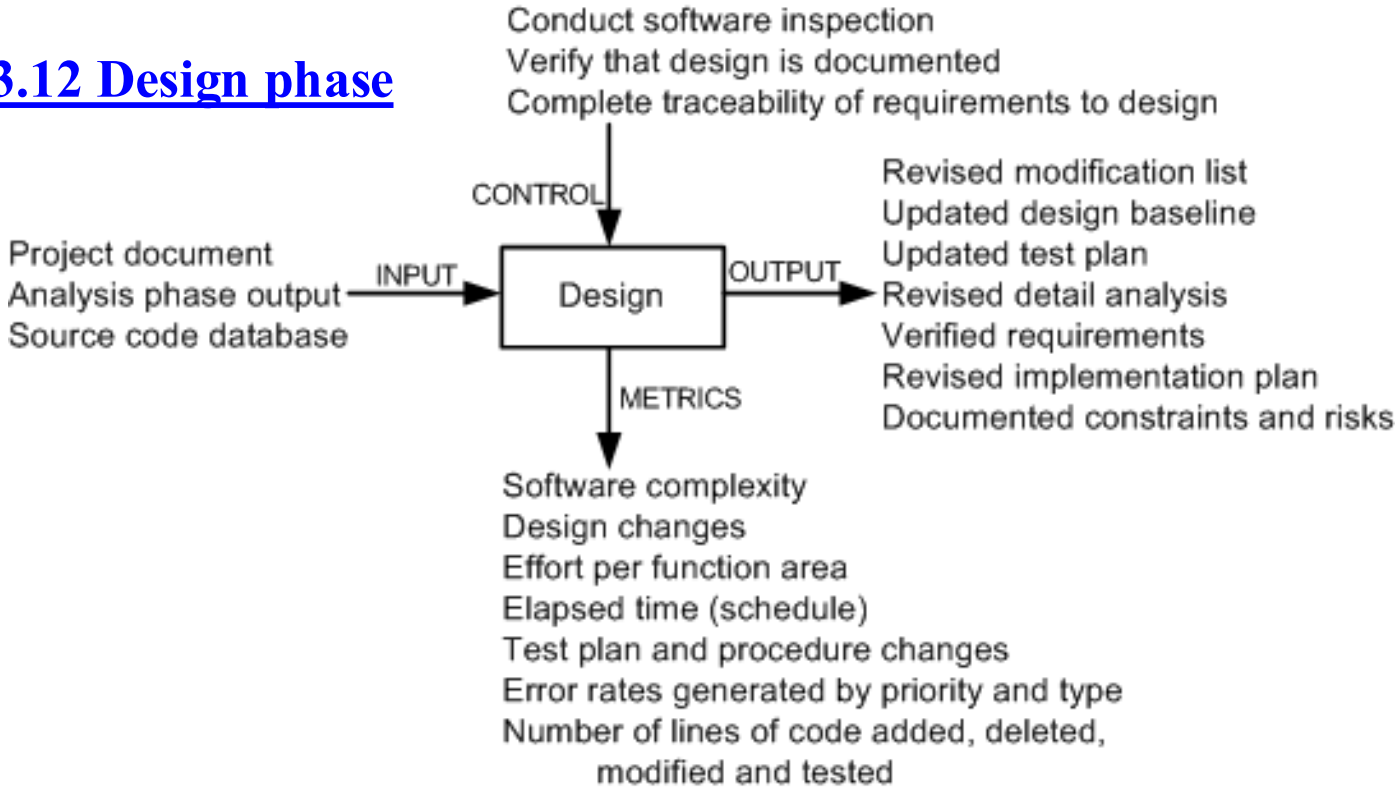
(ii) detailed analysis.

- First, feasibility analysis is performed to: (i) determine the impact of the change; (ii) investigate other possible solutions including prototyping; (iii) assess both short-term and long-term costs; and (iv) determine the benefits of making the change.
- The second phase identifies: (i) firm modification requirements; (ii) the software components involved; (iii) an overall test strategy; and (iv) an implementation plan.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

Figure 3.12 Design phase



Activities of this phase are as follows:

(i) identify the affected software components.

(ii) modify the software components.

(iii) document the changes.

(iv) create a test suite for the new design.

(v) select test cases for regression testing.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

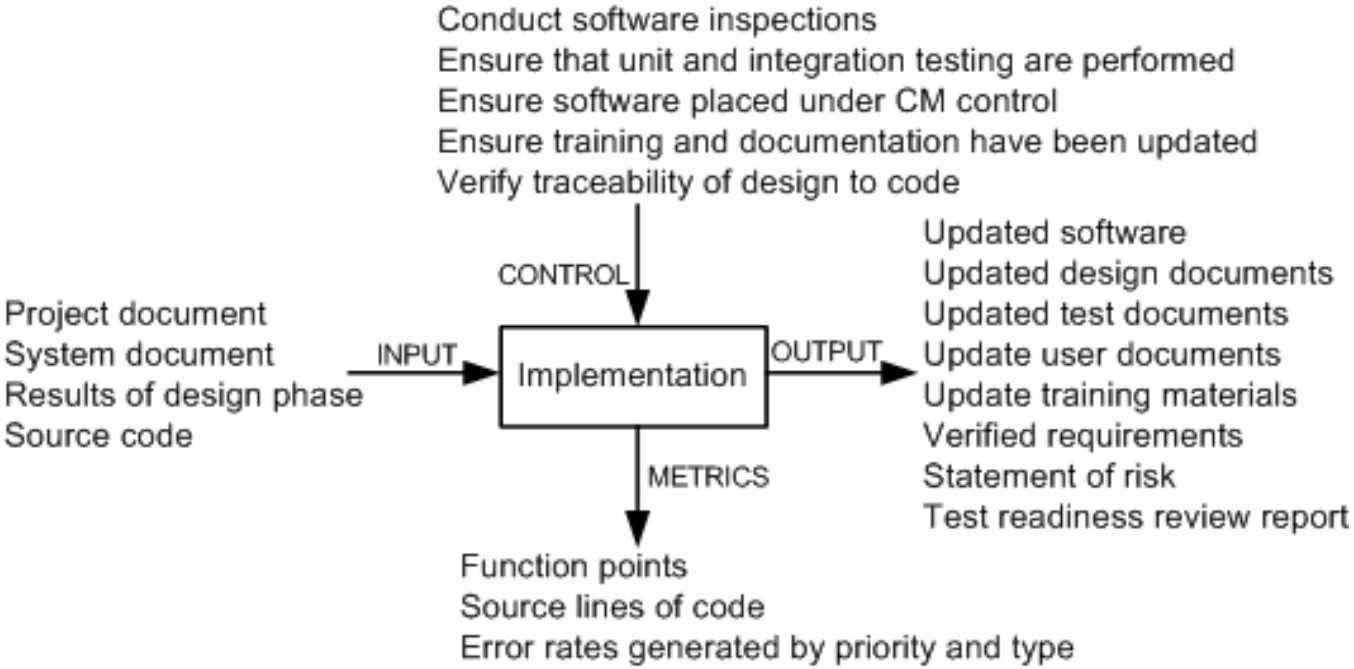


Figure 3.13 Implementation phase

The activities executed in this phase are:

- writing new code and performing unit testing,
- integrating changed code,
- conducting integration and regression testing,
- performing risk analysis, and
- reviewing the system for test-readiness.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

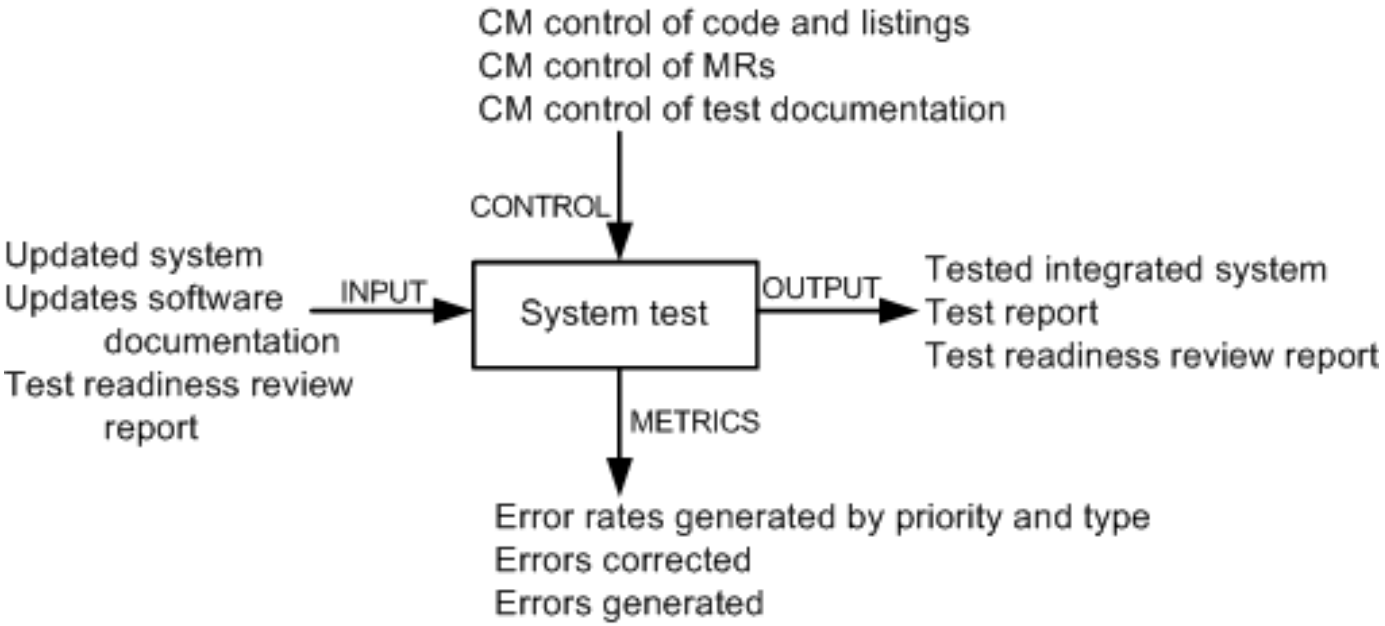


Figure 3.14 System test phase

In this phase tests are performed on the full system to ensure that the modified system complies with the original requirements as well as the new modifications.

System-level testing comprises a broad spectrum of testing activities: functionality testing, robustness testing, stability testing, load testing, performance testing, security testing, and regression testing.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

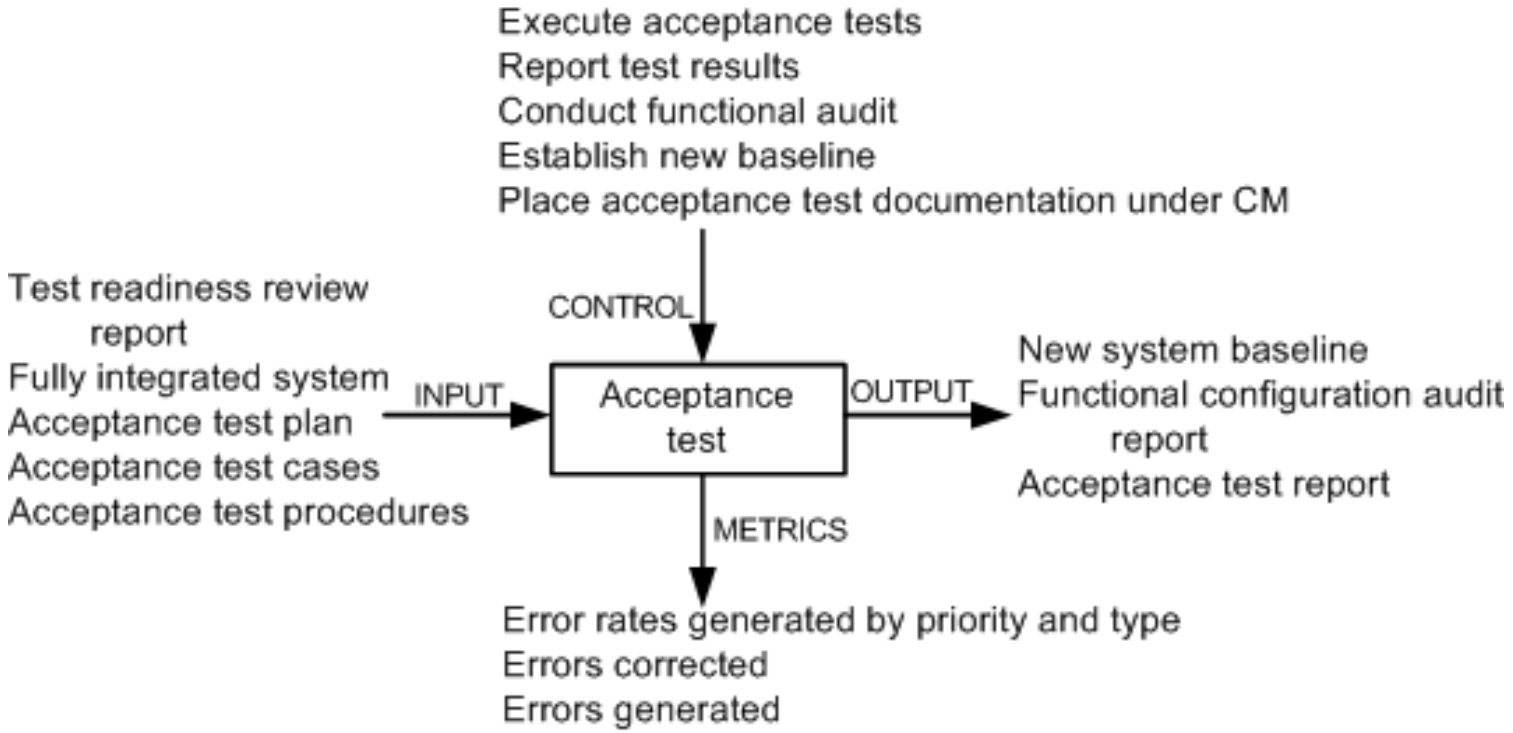


Figure 3.15 Acceptance test phase

- Acceptance testing is performed on a completely integrated system, and it involves customers, users, or their representatives.
- The main objective of acceptance testing is to assess the overall quality of the system, rather than actively identify defects.
- An important concept in acceptance testing is the customer's expectation from the system.

© Tripathy & Naik

3.6 IEEE/EIA 1219 Maintenance Process

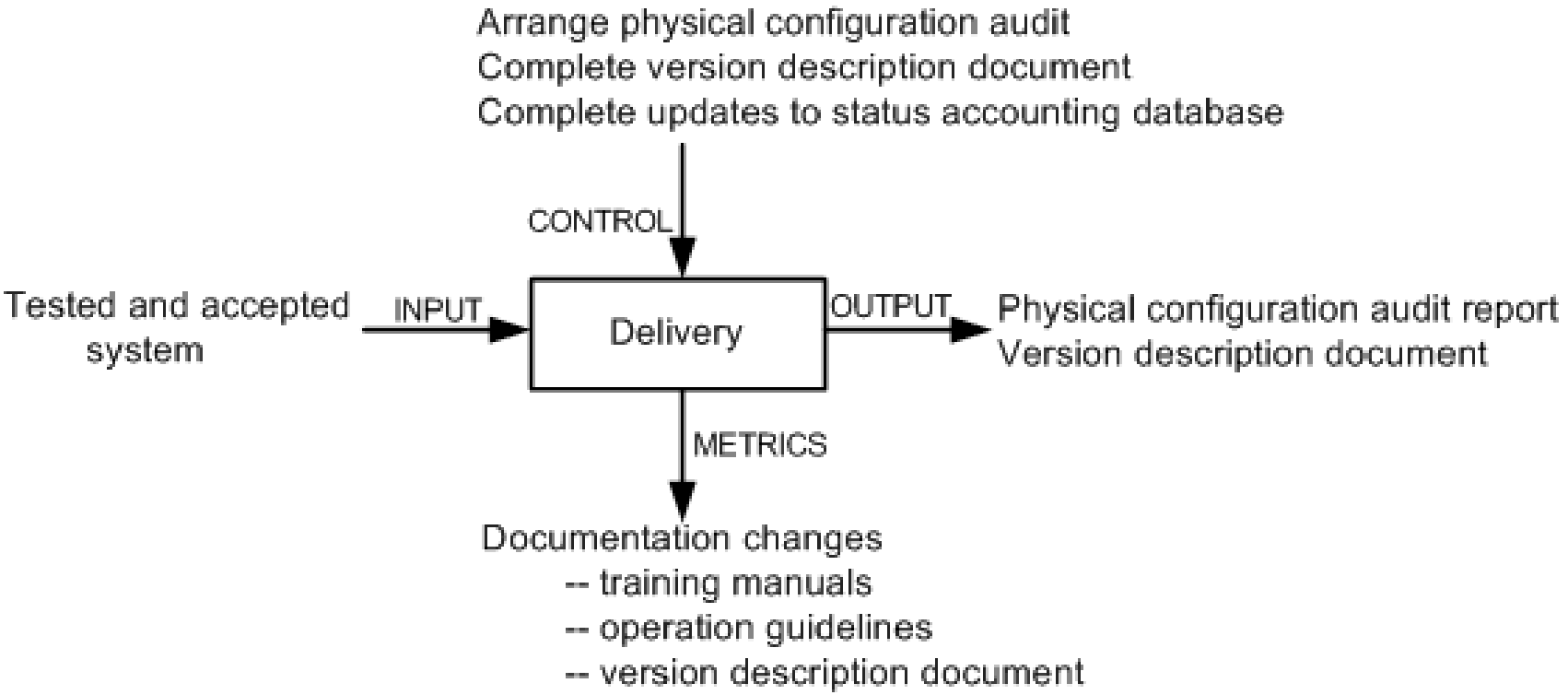


Figure 3.16 Delivery phase

- In this phase, the changed system is released to customers for installation and operation.
- Included in this phase are the following activities: notify the user community, perform installation and training, and develop an archival version of the system for backup.

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

- ISO/IEC 14764 is an international standard for software maintenance.
- It describes maintenance using the same concepts as IEEE/EIA 1219 except that they are depicted slightly differently.
- The basic structure of an ISO process is made up of activities, and an activity is made up of tasks.
- Upon an activation of the maintenance process, plans and procedures are developed and resources are allocated to carry out maintenance.
- In response to a change request, code is modified in conjunction with the relevant documentation.
- Modification of the running software without losing the system's integrity is considered to be the overall objective of maintenance.

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

The maintenance process comprises the following high level activities:

1. Process Implementation.
2. Problem and Modification Analysis.
3. Modification Implementation.
4. Maintenance Review and Acceptance.
5. Migration.
6. Retirement.

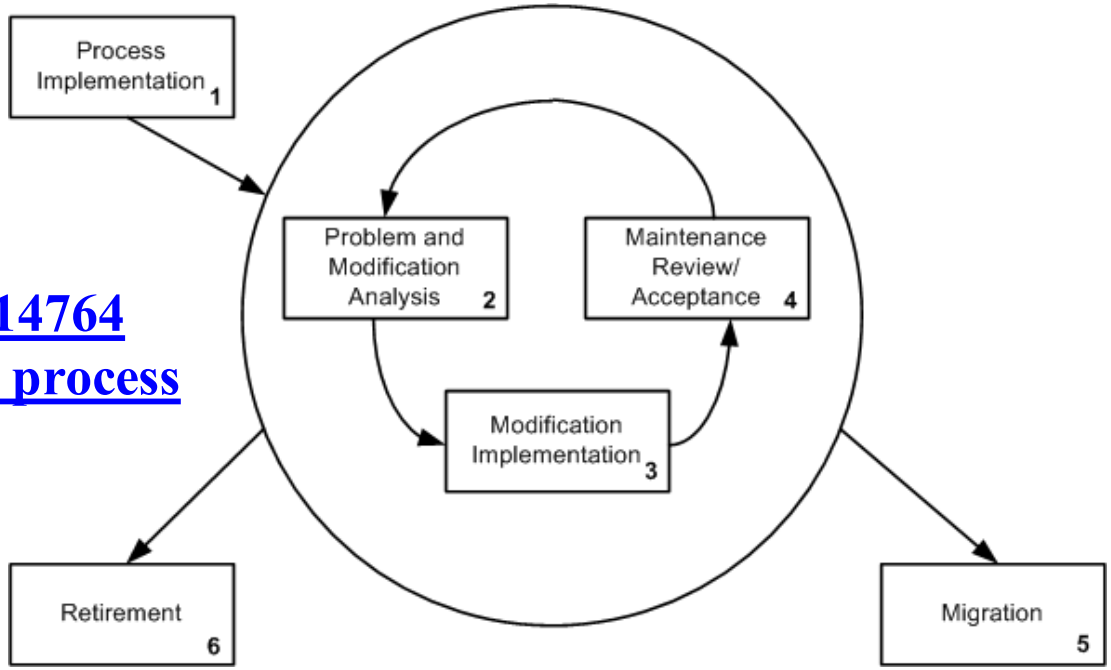


Figure 3.17 ISO/IEC 14764 iterative maintenance process ©IEEE, 2004

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

The process implementation activity consists of four major tasks:

- Develop Maintenance Plans and Procedures
- Establish Modification Request (MR) /Problem Report (PR) Procedures
- Implement Configuration Management
- Develop a Configuration Management Plan

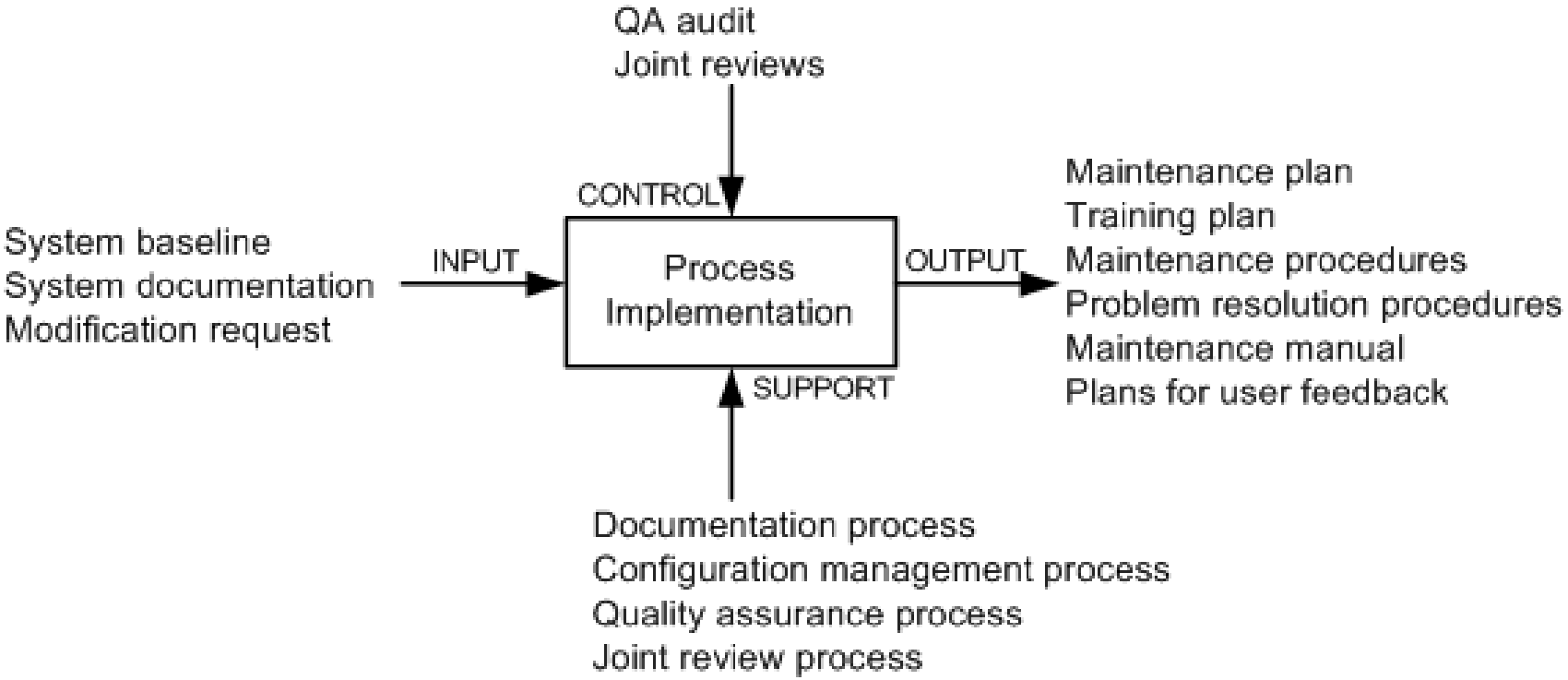


Figure 3.18 Process implementation activity

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.1 Template of a Maintenance Plan

1. Introduction
This section outlines the goals, purpose, and general scope of the maintenance effort.
Also, deviations from the standard are identified.

2. References
The documents that impose constraints on the maintenance effort are identified in this section.
In addition, other documents supporting maintenance activities are identified.

3. Definitions
All terms required to understand the maintenance plan are defined in this section.
If some terms are already defined in other documents, then references are provided to those documents.

4. Software Maintenance Overview
This section briefly describes the following aspects of the maintenance process:
4.1 Organization
4.2 Scheduling Priorities
4.3 Resource Summary
4.4 Responsibilities
4.5 Tools, Techniques, and Methods

5. Software Maintenance Process
This section describes the actions to be executed in each phase of the maintenance process.
Each action is described in the form of input, output, process, and control.
5.1 Problem Identification/Classification and Prioritization
5.2 Analysis
5.3 Design
5.4 Implementation
5.5 System Testing
5.6 Acceptance Testing
5.7 Delivery

6. Software Maintenance Reporting Requirements
This section briefly describes the process for gathering information and disseminating it to members of the maintenance organization.

7. Software Maintenance Administrative Requirements
Describes the standard practices and rules for anomaly resolution and reporting.
7.1 Anomaly Resolution and Reporting
7.2 Deviation Policy
7.3 Control Procedures
7.4 Standards, Practices, and Conventions
7.5 Performance Tracking
7.6 Quality Control of Plan

8. Software Maintenance Documentation Requirements
Describes the procedures to be followed in recording and presenting the outputs of the maintenance process.

Source: From Reference 25. © 1998 IEEE.

UNCORRECTED PROOFS

3.7 ISO/IEC 14764 Maintenance Process

The problem and modification analysis activity comprises five tasks:

- MR/PR Analysis
- Problem replication or verification
- Development of options for implementing the modification
- Documentation of MR/PR, results, and execution options
- Approval of the selected modification option

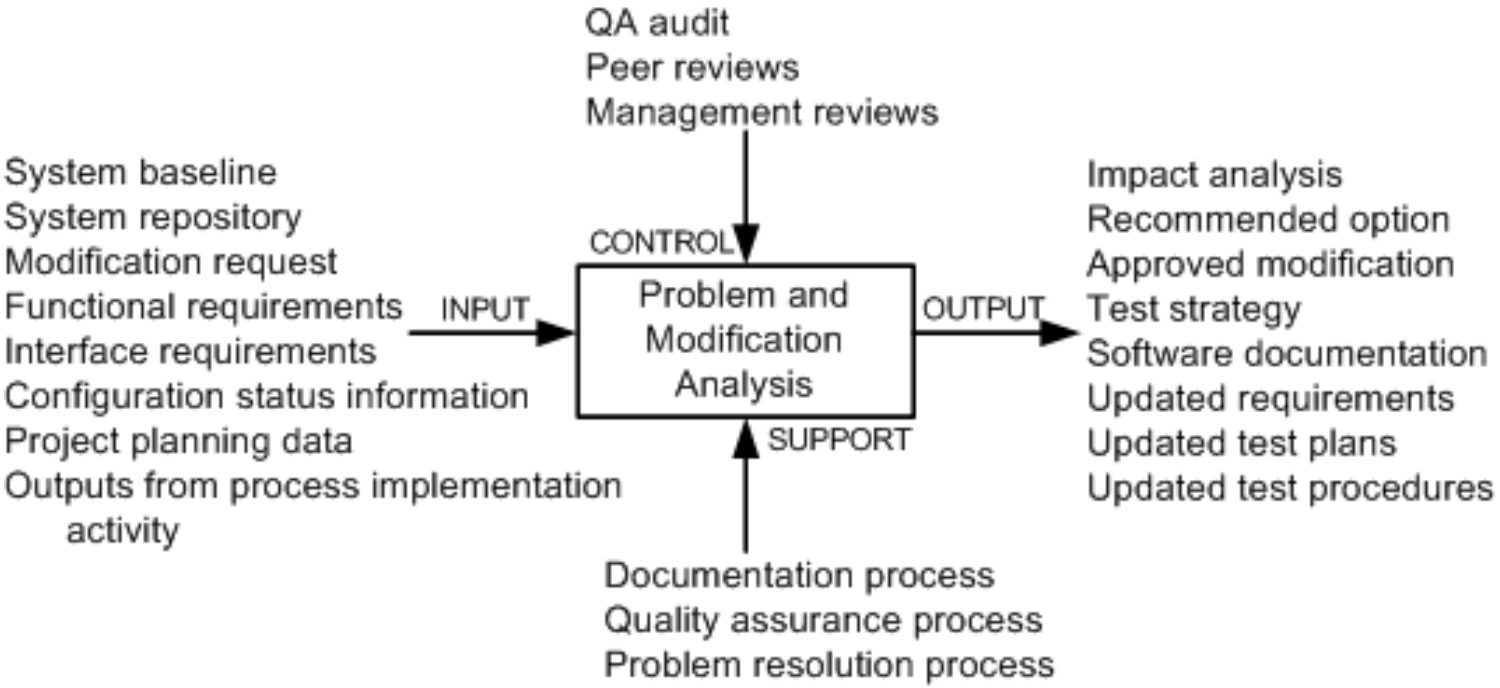


Figure 3.19 Problem modification activity

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.2 Modification Request Task Steps

1. Decide whether or not the maintainer is adequately staffed to make the proposed changes.
2. Decide whether or not the maintenance program has received adequate budget.
3. Decide whether or not enough resources are available and whether the proposed change will effect some current or future projects.
4. Determine the operational issues to be considered.
5. Determine handling priority.
6. Classify the type of maintenance.
7. Determine the impact to current and future users.
8. Determine safety and security implications.
9. Identify ripple effects.
10. Determine any hardware or software constraints that may result from the proposed changes.
11. Estimate the values of the benefits of making the changes.
12. Determine the impact on existing schedules.
13. Document the risks resulting from the impact analysis.
14. Estimate the evaluation to be performed.
15. Estimate the cost of management to execute the modification.
16. Place developed artifacts under CM.

ED PROOFS

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.3 Option Task Steps

1. The MR is assigned a work priority.
2. Explore a work-around for the problem. If a work-around exists, provide it to the user.
3. Identify concrete requirements for the planned modification.
4. Calculate the magnitude and size of the planned modification.
5. Identify a variety of options to execute the planned modification.
6. Estimate the impacts of the options on the users and system hardware.
7. Analyze the risks of each option.
8. Document the outcomes of risk analysis for each of the proposed options.
9. Develop a widely acceptable plan to implement the modification.

TABLE 3.4 Documentation Task Steps

1. Ensure result analyses have been completed and documentations updated. If documentations do not exist, develop new documentation.
2. For accuracy, review the planned strategy to perform tests and review the schedule.
3. Review resource estimates for accuracy.
4. Revise the database for storing accounting status.
5. Describe a procedure to decide whether or not to approve the MR.

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

In this activity, maintainers:

(i) identify the items (including the documentation) to be modified, and

(ii) execute a development process to actually implement the modifications.

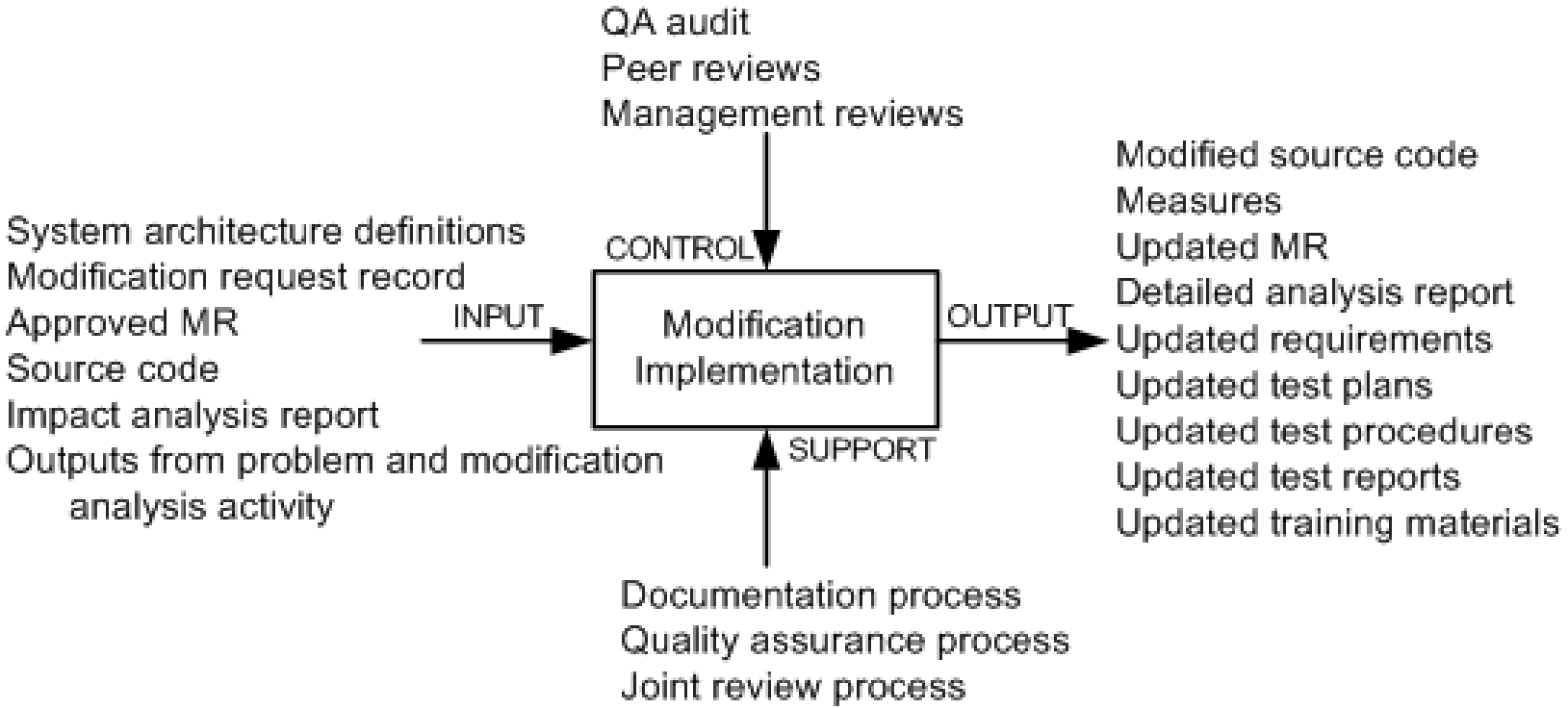


Figure 3.20 Modification implementation activity

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

By means of this activity, it is ensured that:

(i) the changes made to the software are correct, and

(ii) changes are made to the software according to accepted standards and methodologies.

The activity is augmented with the following processes:

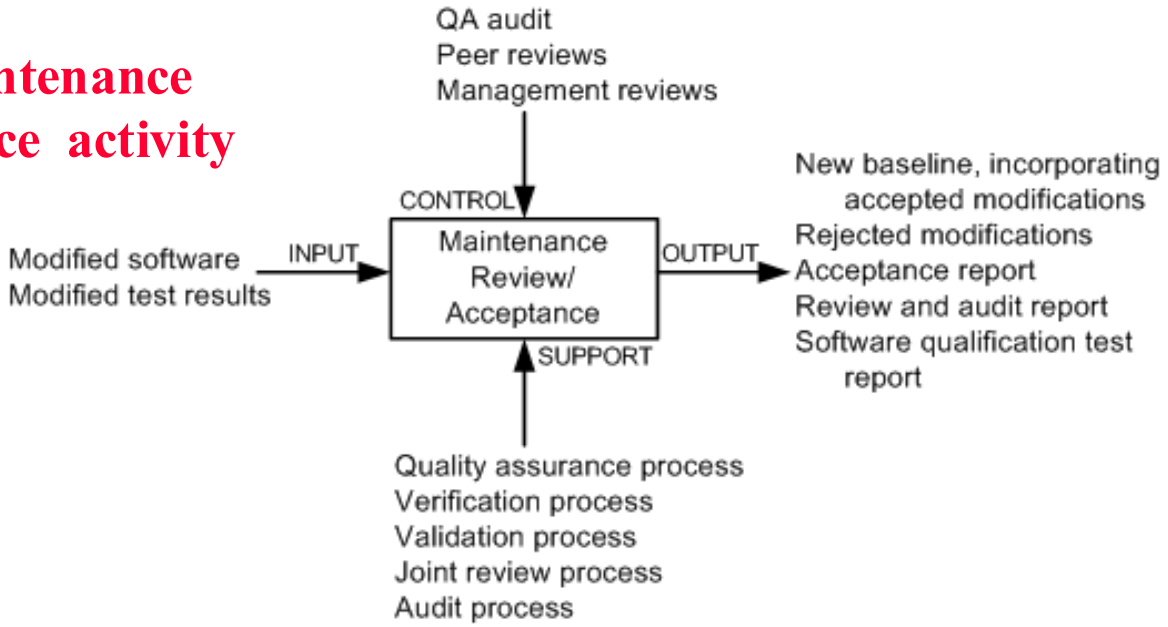
(i) a process for quality management,

(ii) a process to verify the product,

(iii) a process to validate the product, and

(iv) a process to review the product.

Figure 3.21 Maintenance review/acceptance activity



© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.5 Review and Approval Task Steps

Review Task Steps

1. Track the MRs from requirement specification to coding.
2. Ensure that the code is testable.
3. Ensure that the code conforms to coding standards.
4. Ensure that only the required software components were changed.
5. Ensure that the new code is correctly integrated with the system.
6. Ensure that documentations are accurately updated.
7. CM personnel build software items for testing.
8. Perform testing by an independent test organization.
9. Perform system test on a fully integrated system.
10. Develop test report.

Approval Task Steps

1. Obtain quality assurance approval.
2. Verify that the process has been followed.
3. CM prepares the delivery package.
4. Conduct functional and physical configuration audit.
6. Notify operators.
7. Perform installation and training at the operator's facility.

UNCORRECTED

© Tripathy & Naik

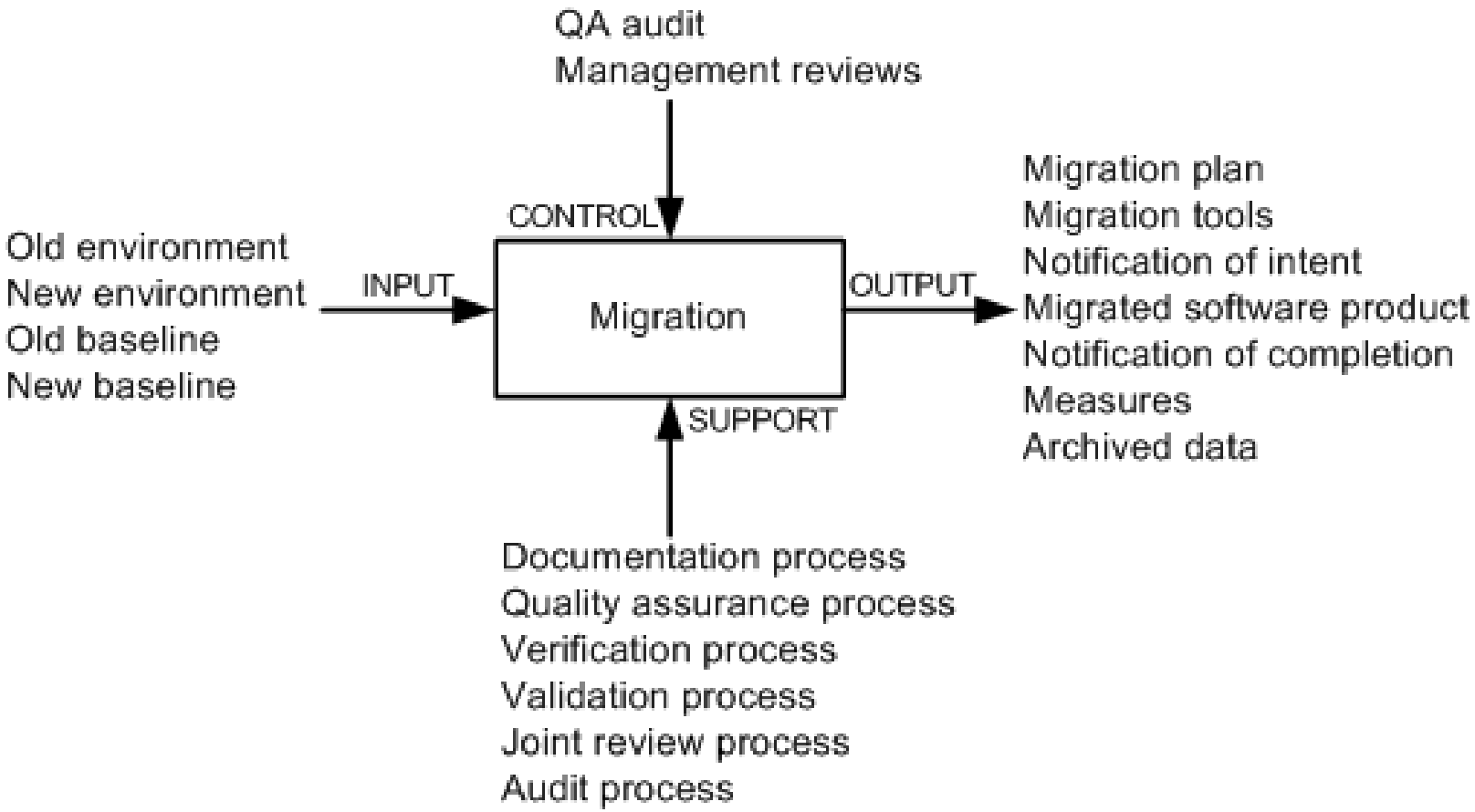
3.7 ISO/IEC 14764 Maintenance Process

Migration is effected in two broad phases:

(i) identify the actions required to achieve migration, and

(ii) design and document the concrete steps to be executed to effect migration.

Figure 3.22 Migration activity



© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.6 Migration Plan Task Steps

1. Analyze the requirements for migration.
2. Perform an impact analysis of migrating the software system.
3. Make a schedule to execute migration.
4. Determine all requirements for data collection to perform post-operation review.
5. Identify and record the migration effort.
6. Identify and reduce risks.
7. Identify the required tools to support migration.
8. Determine how the old environment is going to be supported.
9. Acquire and/or design new tools to support migration.
10. Partition software products and data for conversion in an incremental manner.
11. Prioritize the activities involving data conversion and software products.
12. Execute software products and data conversions.
13. Perform migration of software products and data to the new environment.
14. Operate the migrated system and the old system in parallel as much as possible.
15. Perform testing to ensure the success of migration.
16. Should there be a need, continue to provide support for the old environment.

PROOF

© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.7 Operation and Training Task Steps

Parallel Operations Task Steps

1. Survey the site.
2. Install hardware equipment.
3. Install the software system.
4. Run basic tests to ensure that hardware and software have been correctly installed.
5. Run both the new and old systems in parallel, under the desired operational load.
6. Gather data from the old and the new systems.
7. Analyze the collected data.

Training Task Steps

1. Identify the requirements for migration training.
2. Schedule the requirements for migration training.
3. Review the migration training.
4. Update the plan to provide training.

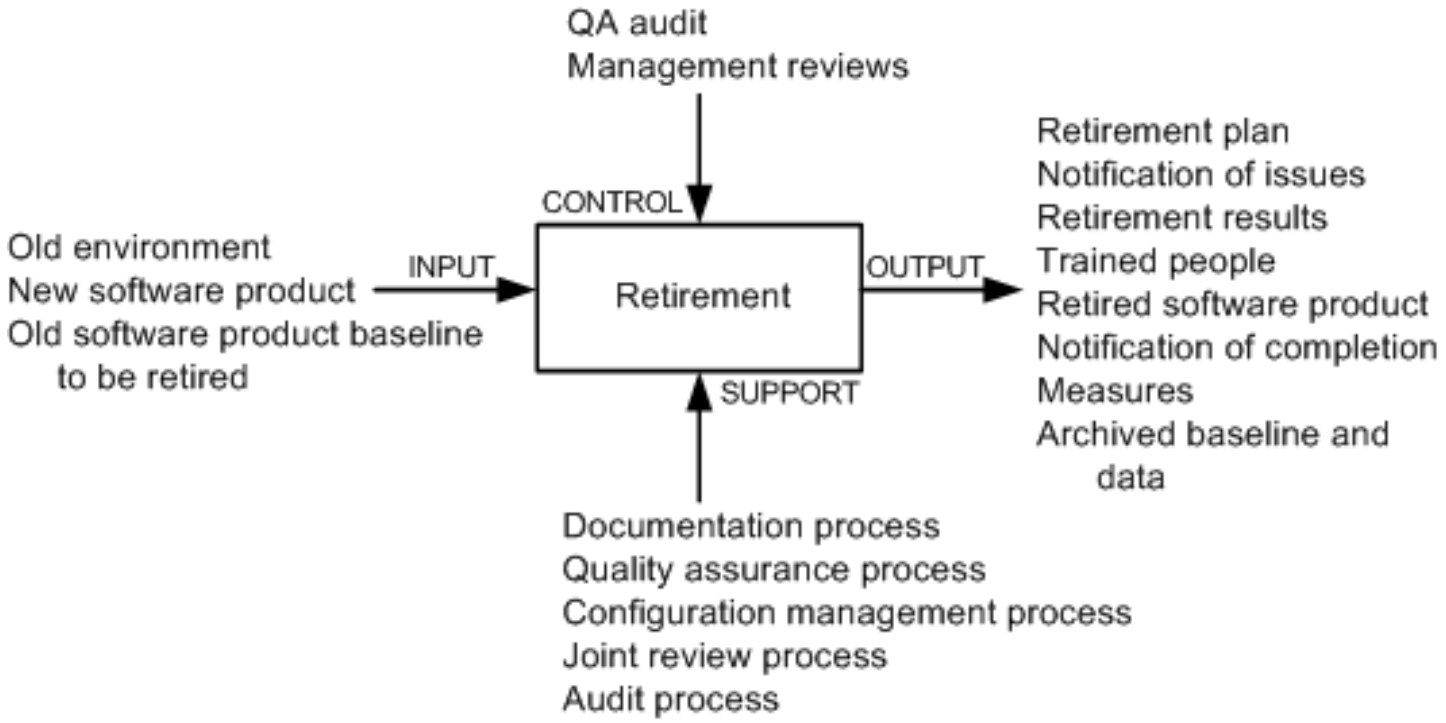
© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

This activity comprises five tasks:

- Retirement plan
- Notification of intent
- Implement parallel operations and training
- Notification of completion
- Data archival

Figure 3.23 Retirement activity



© Tripathy & Naik

3.7 ISO/IEC 14764 Maintenance Process

TABLE 3.8 Retirement Plan Task Steps

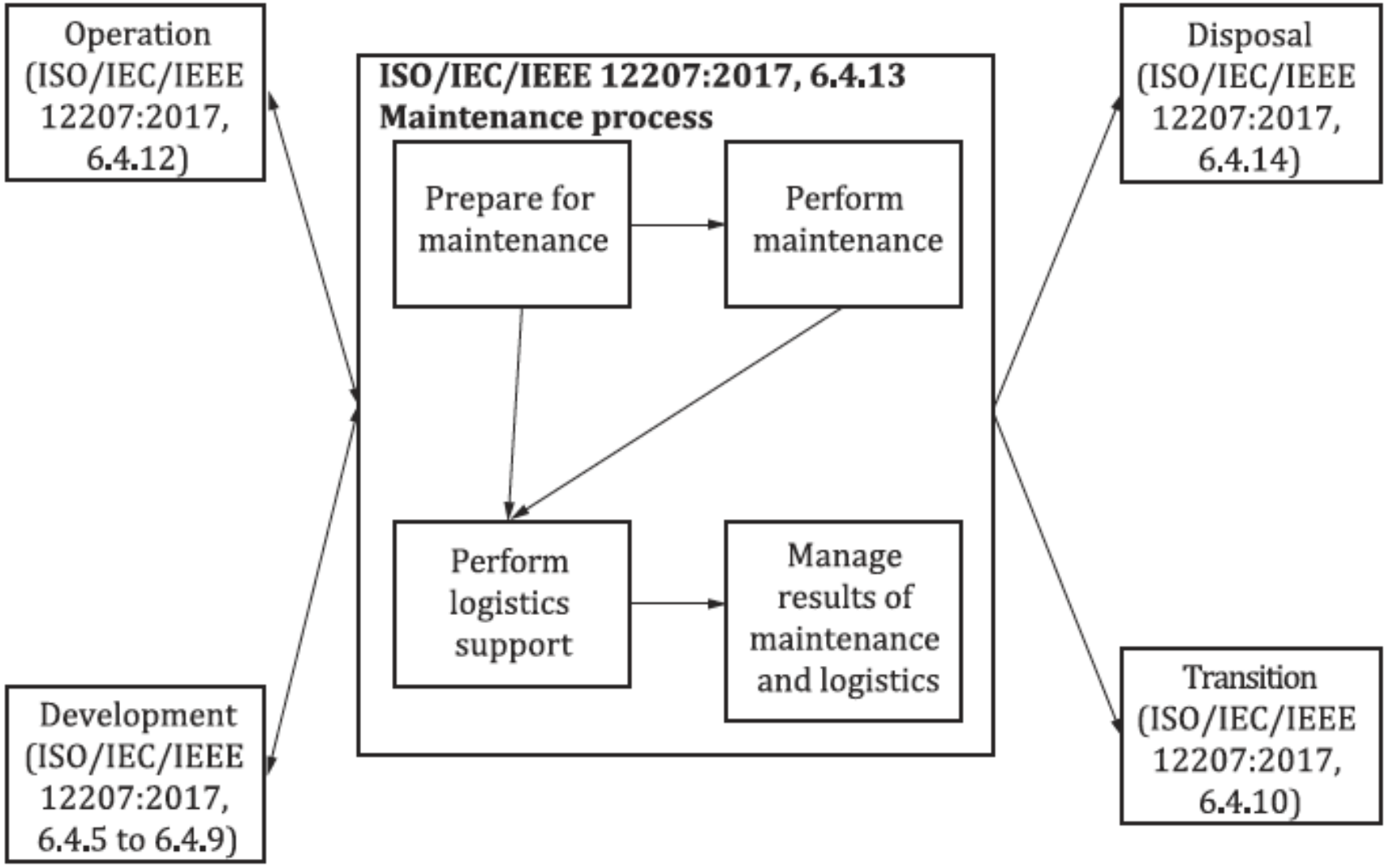
1. Analyze the requirements to retire the systems.
2. Determine what impacts the retiring software will have.
3. Identify a product that will replace the software to be retired.
4. Make a schedule to retire the software.
5. Determine the need for residual support in the future.
6. Identify and describe the retirement effort.

© Tripathy & Naik

ISO/IEC/IEEE 14764: 2022

- Maintenance is one of the technical life cycle processes that may be performed during the life cycle of software (ISO/IEC/IEEE 12207).
- The acquisition and supply agreement life cycle processes of ISO/IEC/IEEE 12207 may initiate the process implementation activity of the maintenance (life cycle) process through an agreement or contract.
- The operation process may call for maintenance through a modification request or problem report.
- The maintenance (life cycle) process invokes the technical life cycle processes of ISO/IEC/IEEE 12207 to develop any required enhancement, as a part of the maintenance strategy.
- The technical management processes, such as project planning, configuration management and quality assurance of ISO/IEC/IEEE 12207 are used by the maintenance (life cycle) process.

Maintenance process and relationship with other processes of ISO/IEC/IEEE 12207



3.8 Software Configuration Management

- The concept of configuration management (CM) was developed to manage changes in large systems.
- It handles the control of all product items and changes to those items.
- Software Configuration Management (SCM) is applied to software products.
- The product items include document, executable software, source code, hardware, and disks.
- SCM accrues two kinds of benefits to an organization:

  SCM ensures that development processes are traceable and systematic so that all changes are precisely managed.

  SCM enhances the quality of the delivered system and the productivity of the maintainers.
- The objectives of SCM are to:
  - Uniquely identify every version of every software at various points in time.
  - Retain past versions of documentations and software.
  - Provide a trail of audit for all modifications performed.
  - Throughout the software life-cycle, maintain the traceability and integrity of the system changes.

© Tripathy & Naik

3.8 Software Configuration Management

Projects benefit from effective SCM as follows:

1. Confusion is reduced and order is established.
2. To maintain product integrity, the necessary activities are organized.
3. Correct product configurations are ensured.
4. Quality is ensured – and better quality software consumes less maintenance efforts.
5. Productivity is improved, because analysts and programmers know exactly where to find any piece of the software.
6. Liability is reduced by documenting the trail of actions.
7. Life cycle cost is reduced.
8. Conformance with requirements is enabled.
9. A reliable working environment is provided.
10. Compliance with standards is enhanced.
11. Accounting of status is enhanced.

© Tripathy & Naik

3.8.1 Brief History

- A need for configuration management was originally felt in the aerospace industry in the 1950s.
- In the 1970s, large scale computer software began to pose many of those same change management problems.
- Software maintenance engineers borrowed configuration management techniques from the aerospace industry to manage software modifications.
- In the beginning, punch cards with different colors were used to indicate changes.
- In the late 1960s, to indicate changes to the UNIVAC-1100 EXEC-8 operating system, maintenance personnel used “corrective cards.”
- At that time, development of operating systems benefited from SCM.

© Tripathy & Naik

3.8.1 Brief History

- In the 1970s and the 1980s, SCM emerged as a distinct discipline.
- the Unix-based software tool Make accepts descriptions of system configurations, and can automatically construct the system from its descriptions.
- in the 1980s, delta algorithms based on text matching were developed to enable SCM tools to store just the differences among versions.
- Consequently, novel algorithms were developed for efficient storage and retrieval of non-textual objects.
- These days SCM systems support the management of evolution of a broad range of software systems that are being modified by a large number of maintenance personnel working in different countries and utilizing a variety of machines.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

- Estublier et al. classified the functionalities into three broad areas:
  - product,
  - tool, and
  - Process.

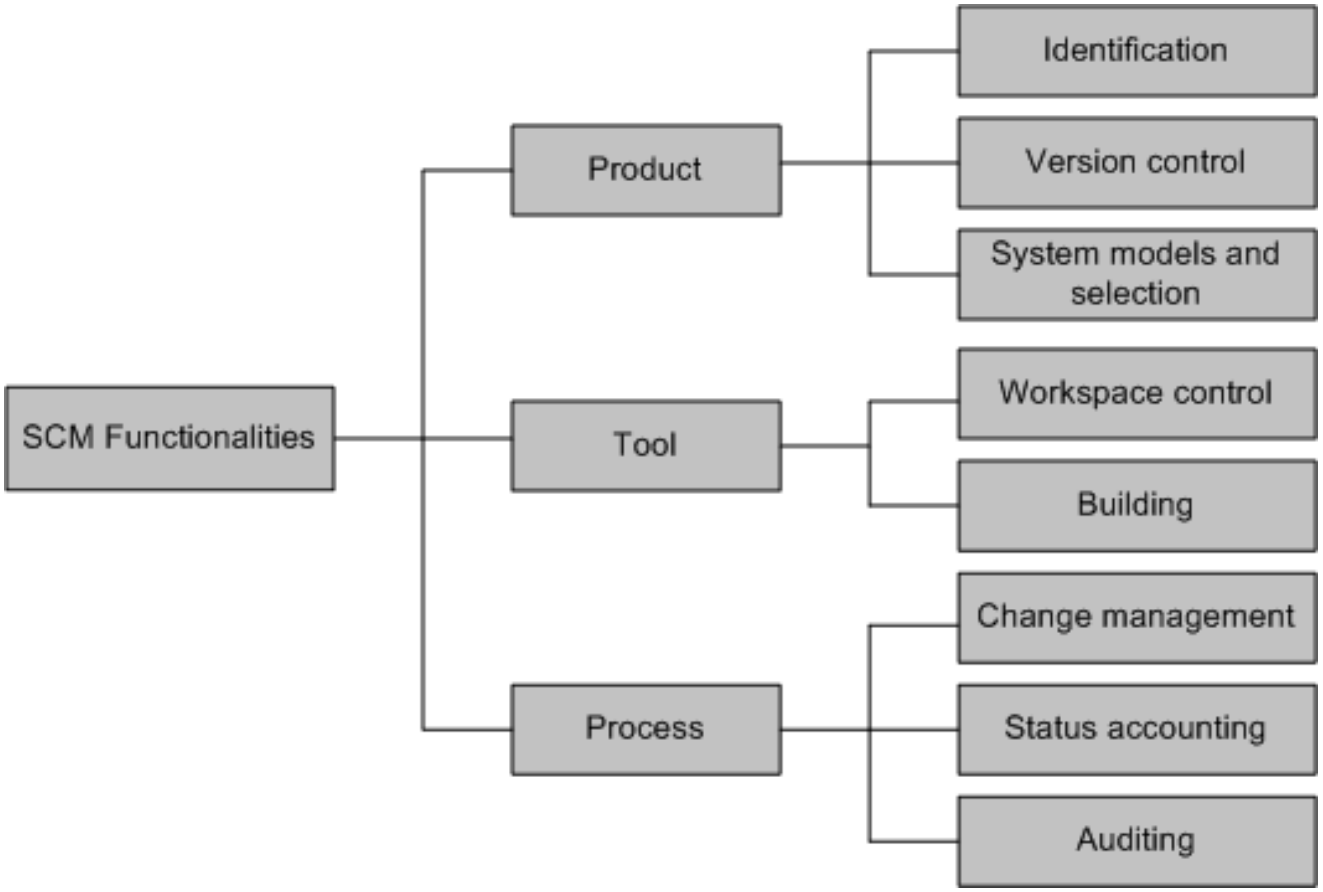


Figure 3.24 Technical dimension of SCM systems

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Identification

- The items whose configurations need to be managed are identified in this function.
- The identified items include specification, design, documents, data, drawings, source code, executable code, test plan, test script, hardware components, and components of the software development environment, namely, compilers, debuggers, and emulators.
- Project plan and customer requirements should also be included.
- To accurately identify products, including their configuration and version levels, a schema of names and numbers is designed.
- Finally, for all configuration items and systems, a baseline configuration is established.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Version control

- To avoid confusion during the process of artifact evolution, a new identifier is assigned to the artifact every time the artifact is modified.
- One may be interested in recording a fact that a given artifact fixes a subset of defects found in an earlier release.
- The aforementioned kind of relation can be recorded by means of the version control (VC) functionality of SCM by:

  (i) interpreting software artifacts as configuration items, and

  (ii) identifying the relations, if there is any, among the configuration items.
- The basic version control idea is to have two separate files: master copies and working copies.
- The former is stored in a centralized repository.
- Software developers check out working copies from the repository, modify the working copies, and, finally, check in the working copies into the repository.
- Checking in a file means committing to the changes made to the working copies.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Version control

- Conflicts can arise if many software developers want to use the same version of a file.
- Conflicts can be resolved by means of two techniques: lock-modify-unlock and copy-modify-merge.
- Version control must support parallel development by allowing branching of versions.
- Consider the scenario: (i) an organization is currently developing the next version of their already released application; and (ii) a report about a major defect is received from the end users.
- Now the development group has the option to retrieve the released version and create a branch, as illustrated in Figure, to fix the defect.
- The figure illustrates how a file evolves with two branches, where the main path is called trunk. As shown in the figure 3.25, branch changes are incorporated by merging with the trunk.

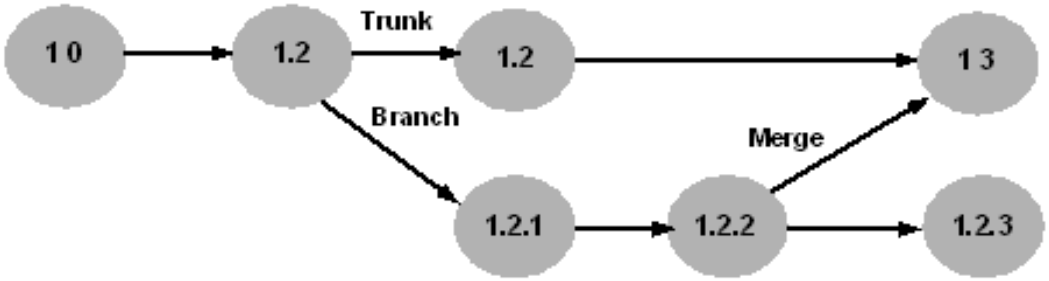


Figure 3.25 An evolution of a file with two branches

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

System models and selections

- It is neither efficient nor effective to manage a project file-by-file.
- There is a need to support aggregate artifacts so that maintenance personnel can enforce consistency in large projects by means of relationships among artifacts and attributes.
- Relationships among artifacts and attributes are captured by developing models which support the idea of software configurations.
- Intuitively, a configuration means an aggregate of versionable items.
- The general idea of configuration raises a need for enabling users to have selective access to parts and versions of such aggregated artifacts.
- By default, SCCS (Souerce Code Control System) and RCS (Revision Control System) keep in the workspace the most recent version of the principal variant.
- Next, all artifacts that are exceptions to the default placement rule can be explicitly fetched by the user.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Workspace control

- An environment that enables the maintainer to make and test the changes in an isolated manner is called workspace.
- In an SCM system, software versions are stored in a repository that can not be directly modified. Rather, when a need to modify some files arises, the files are copied into a workspace.
- One can realize a workspace in two ways:

  (i) it can be as simple as the home directory of the programmer who wants to modify the files;

  (ii) it can be a complex mechanism such as an integrated development environment and a database.
- In general, three basic functions are performed in a workspace:
  - **Sandbox:** Checked out files are put in a workspace to be freely edited. In addition, it is not necessary to lock the original files in the repository.
  - **Building:** An SCM system generally stores the differences between successive versions to save space. Therefore, the workspace expands the deltas into full-fledged source files. In addition, the workspace stores the derived binaries.
  - **Isolation:** Every developer maintains at least one workspace. Therefore, the developer makes modifications to the source code, compiles the files, performs tests, and debugs code without impacting the works of other developers.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Building

- Efficiency is a key requirement of SCM systems so that developers can quickly build an executable file from the versioned source files.

- Second requirement of SCM systems is that it must enable the building of old versions of the system for recovery, testing, maintenance, or additional release purpose.

- Finally, SCM systems must support building of software.
- The build process and their products are assessed for quality assurance.
- Outputs of the build process become quality assurance records, and the records may be needed for future reference.

- The ***make*** application on the Unix operating system, originally developed by researchers at Bell Laboratories, is a classical example of a build process.
- Commercial SCM systems such as *ClearCase* continue to rely on variants of ***make.***

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Change management

- SCM systems must:

  (i) enable users to understand the impact of modifications.

  (ii) enable users to identify the products to which a specific modification applies.

  (iii) provide maintenance personnel with tools for change management so that all activities from specifying requirements to coding can be traced.

- In the beginning, CRs were managed in paper form.
- However, these days CRs are saved in the SCM repository and are linked with the actual modifications, in addition to being automated.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Accounting

- The primary purpose of status accounting is to:

  (i) keep formal records of already existing configurations, and

  (ii) produce periodic reports about the status of the configurations.

  These records:

  - describe the product correctly,
  - are used to verify the configuration of the system for testing and delivery, and
  - maintain a history of change requests, including both the approved ones and the rejected ones.

- A history of change request includes the answers to the following questions:
  - Why are changes made?
  - When are the changes made?
  - Who makes the changes?
  - What changes are made?

- Status accounting is useful in communicating important details of the project and configuration items to the stakeholders of the project.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Auditing

- SCM systems need to provide the following features:

(i) roll back to earlier stable points.

(ii) identify which modifications were performed, why those modifications were performed, and who performed those modifications.

- By means of auditing, the organization maintains the integrity of the baselines and release configurations for all products.
- Two kinds of audits are performed before a software is released:

audit for functional configuration: determines whether or not the software satisfies the user requirement specification and the system requirement specification.

audit for physical configuration: It verifies if the reference and design documents accurately represent the software.

© Tripathy & Naik

3.8.2 SCM Spectrum of Functionality

Auditing

- Overall, a **configuration audit** tries to find answers to the following:
  - To what extent are the requirements satisfied by the modified system?
  - Does the software release under consideration reflect the modification requests?
- The activities to perform a **configuration audit** are as follows:
  - Procedures and an audit schedule are defined.
  - The personnel to perform the audits are identified.
  - Established baselines are audited.
  - Audit reports are generated.

© Tripathy & Naik

3.8.3 SCM Process

- Figure 3.26 shows the three major SCM implementation activities:
  - planning,
  - baseline development, and
  - configuration control.

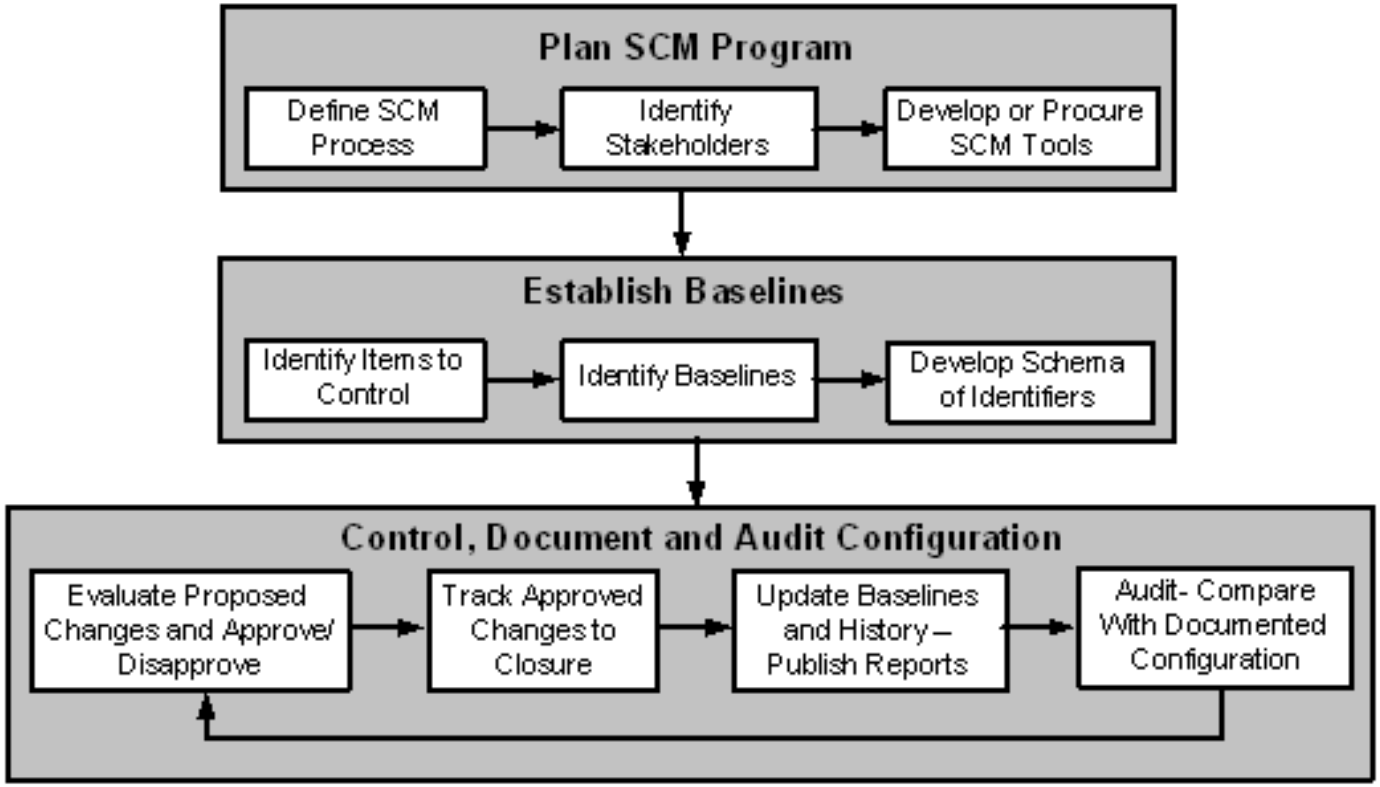


Figure 3.26 A process for implementing SCM

© Tripathy & Naik

3.8.3 SCM Process

Planning

- Planning is begun with two activities:
  - (i) define the SCM process, and
  - (ii) establish procedures.
- A key step during planning is the identification of the stakeholders.
- The stakeholders in a configuration management are the maintainers, development engineers, sustaining test engineers, quality assurance auditors, users, and the management.
- The stakeholders are also known as configuration control board (CCB) members.
- Not all changes are reviewed by the board. Rather, small groups review and approve most of the changes.
- Therefore, those groups need to be identified in the planning phase.
- Various SCM tools are used to maintain configuration history and facilitate the SCM process flow.
- Examples of such tools are CVS (concurrent version system) and Clearcase.

© Tripathy & Naik

3.8.3 SCM Process

Establishing baseline

- Once an SCM program plan is in place, the next step is identification of items (code, data, and documents) that are the subject of configuration control.
- After the configuration items identified, a software baseline library is established to make the set of configurable items publicly available.
- The library, called repository, is the heart of the SCM system.
- The repository has information about all the baselined items.
- The process of baseline (or re-baseline for a change) involve the following activities:
  1. Create a snapshot of the current version of the product and its configuration items, and allocate a configuration identifier to the entire configuration.
  2. Allocate version numbers to the configuration items and check in the configuration.
  3. Store the approved authority information as part of meta data in the repository.
  4. Broadcast all the above information to the stakeholders.
  5. To accurately identify the configuration version, design a schema of words, numbers, or letters for common types of configuration items. In addition, project requirements may dictate specific nomenclature.

© Tripathy & Naik

3.8.3 SCM Process

Controlling, documenting and auditing

- After establishing a baseline, it is important to:

  (i) keep the actual and the documented configuration identical;

  (ii) ensure that the baseline complies with a project's configuration described in the requirements document.

- The aforementioned requirements of a baseline are realized by means of a four-step iterative process illustrated in Figure 3.26.

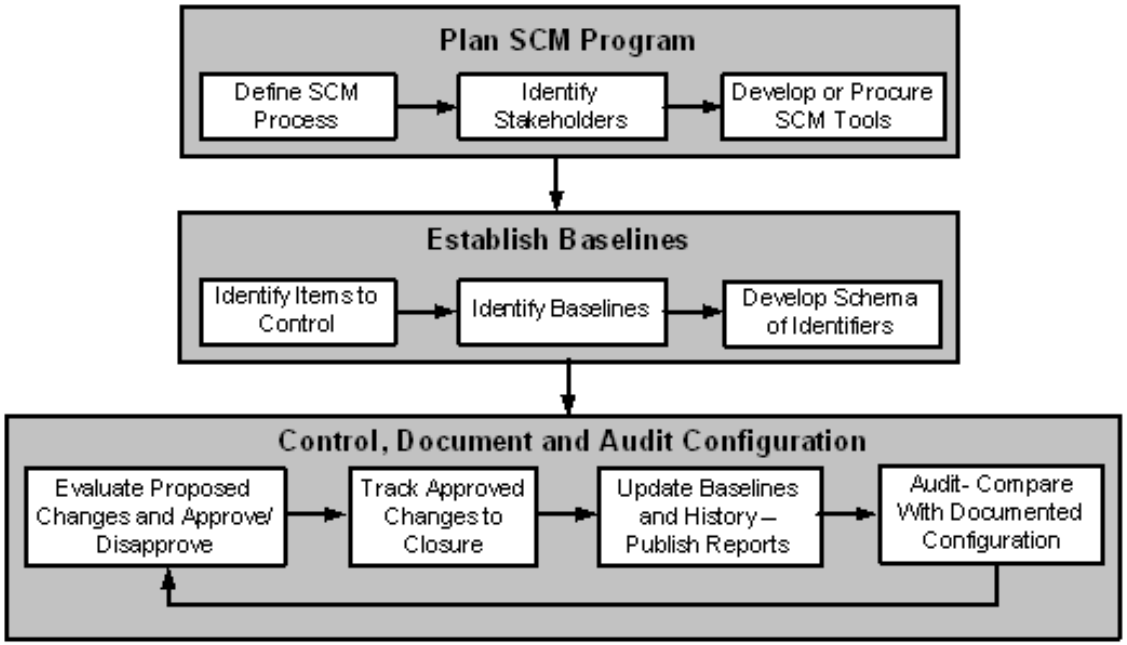


Figure 3.26 A process for implementing SCM

© Tripathy & Naik

3.8.3 SCM Process

Controlling, documenting and auditing

- The stakeholders specified in the SCM plan review and evaluate all changes to the configuration.
- After their evaluations, both approvals and disapprovals are documented.
- Approved changes are tracked until they are verified .
- Next, the appropriate baseline is revised in conjunction with all relevant documents, and reports are generated.
- At regular intervals, records and products are audited to verify that:
  - There is acceptable matching between the documented configuration and the actual configuration.
  - The configuration conforms with the requirements of the project.
  - Documentations of all change activities are complete and up-to-date.
- The three steps in the cycle, namely, controlling, documenting, and auditing, are repeatedly executed throughout the lifetime of the project.

© Tripathy & Naik

3.9 Change Request Workflow

- A change request (CR), also called a modification request (MR), is a vehicle for recording information about a system defect, requested enhancement, or quality improvement.
- Change requests are placed under the control of a change management system.
- Change management systems control changes by an automated system in the form of work-flow.
- The basic objective of change management is to uniquely identify, describe, and track the status of each requested change.
- The objectives of change management are as follows:
  - Provide a common method for communication among stakeholders.
  - Uniquely identify and track the status of each CR. This feature simplifies progress reporting and provides better control over changes.
  - Maintain a database about all changes to the system. This information can be used for monitoring and measuring metrics.

© Tripathy & Naik

3.9 Change Request Workflow

- A change request describes the desires and needs of users which the system is expected to implement.
- While describing a CR, two factors need to be taken into account:
  - Correctness of CRs, and
  - Clear communication of CRs to the stakeholders.
- The results of interpreting a CR in different ways are as follows:
  - The team carrying out actual changes to the system and the team performing tests may develop contradicting views about the new system's quality.
  - The changed system may not meet the needs and desires of the end users.
- CRs need to be represented in an unambiguous manner, and made available in a centralized repository.
- Wide availability of CRs to all the stakeholders is likely to reveal differences in interpretations by different groups.

© Tripathy & Naik

3.9 Change Request Workflow

- The life-cycle of a CR has been illustrated in Figure 3.27, by means of a state-transition diagram.
- Each state represents a distinct stage in the life-cycle of a CR.

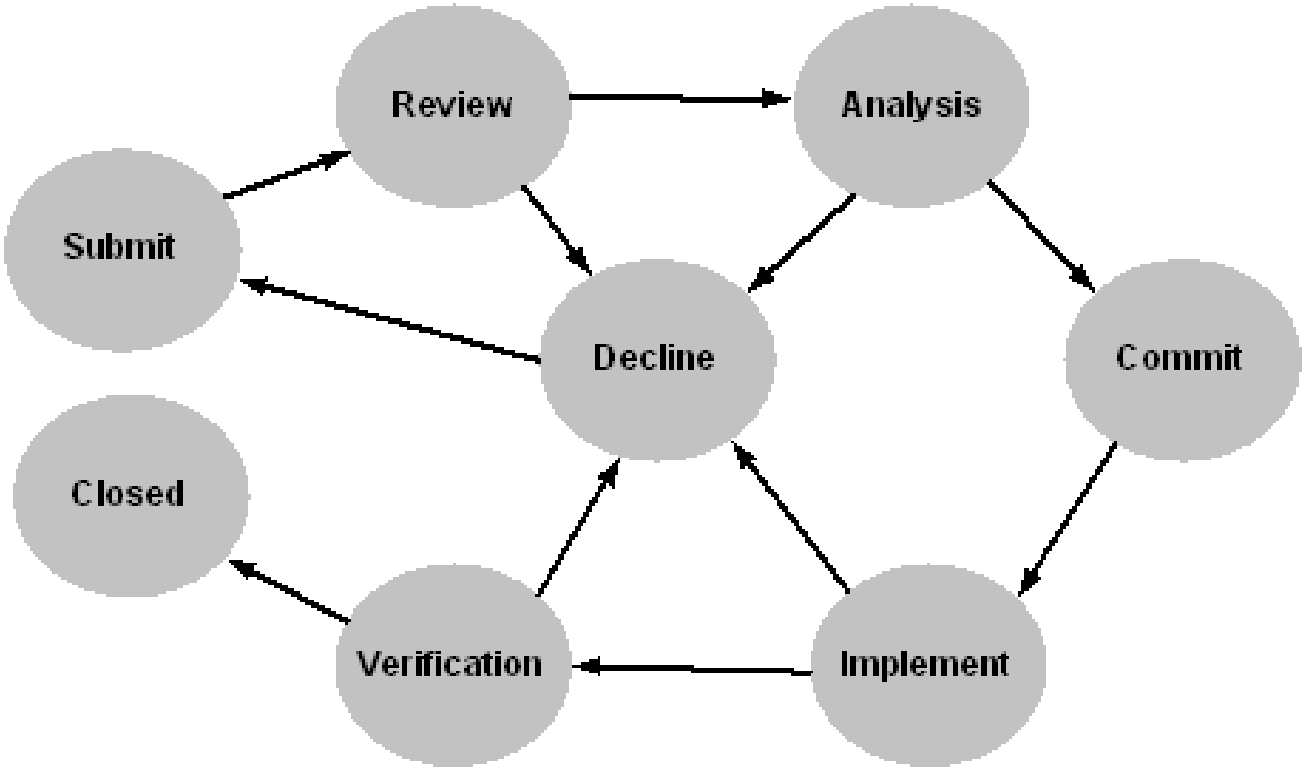


Figure 3.27 State transition diagram of a CR

© Tripathy & Naik

3.9 Change Request Workflow

Field Name	Description
change_request_id	A unique identifier of the CR
title	A concise summary of the CR
description	A short description of the CR
maintenance_type	Classification of the maintenance type in terms of a member of {Corrective, Adaptive, Perfective, Preventive}
product	Product name
component	Component where the change is needed, or where the problem occurred
state	Present state of the CR in terms of a member of {Submit, Review, Analysis, Commit, Implementation, Verification, Closed, Declined}
customer	Name of the customer making the change request
problem_origin	The origin of the problem
impacts	Components that are affected by a change and its ripple effect
resolution	Documentation of what was changed, how, and why
note	Additional information provided by the submitter for subsequent decision making
software_release	The version number of the product release in which the CR is likely to be effective
committed_release	The version number of the product release in which the CR will be effective
priority	Priority of CR, which is an element of a set, namely, {normal, high}
severity	Severity of CR, which is an element of a set, namely, {normal, critical}
marketing_justification	The business justification for the CR to exist
time_to_implement	The time, in person-week, required to effect the change
eng_assigned	The engineering personnel assigned to analyze the CR
functional_spec_title	Title of the specification for functional requirements
functional_spec_name	Name of the file describing the functional requirements
functional_spec_version	This is the most recent version number of the specification of functional requirements
decline_note	The reason for declining the CR
ec_number	The identifier of the engineering change (EC) document
attachment	Attachment to further describe the CR (if any)
tc_id	Identifiers of test cases used in effecting the CR
tc_results	The result of testing: {Untested, Passed, Failed Blocked, Invalid}
verification_method	Record the methods of verification of the CR: analysis, testing, inspection, and/or demonstration
verification_status	The verification state of the CR: Passed, Failed, or Incomplete
compliance	The level of compliance:{Non-compliance, Partial Compliance, or Compliance}
testing_note	Reports from the test personnel, possibly describing the demonstration given to the customers, analysis performed on the change, or inspection of the code performed by test personnel
defect_id	Defect identifier If "Failed" is assigned to the *tc_results* field, the defect identifier is associated with the failed test to indicate the defect causing the failure. The defect identifier is obtained from test database.

Table 3.9 Change request schema field summary

© Tripathy & Naik

Summary

- General Idea
- Reuse Oriented Model
- The Staged Model for CSS
- The Staged Model for FLOSS
- Change Mini-Cycle Model
- IEEE/EIA 1219 Maintenance Process
- ISO/IEC 14764 Maintenance Process
- Software Configuration Management
- Change Request Work-Flow

© Tripathy & Naik